

```

return 0;
}

```

代码运行结果如图 7-21 所示：

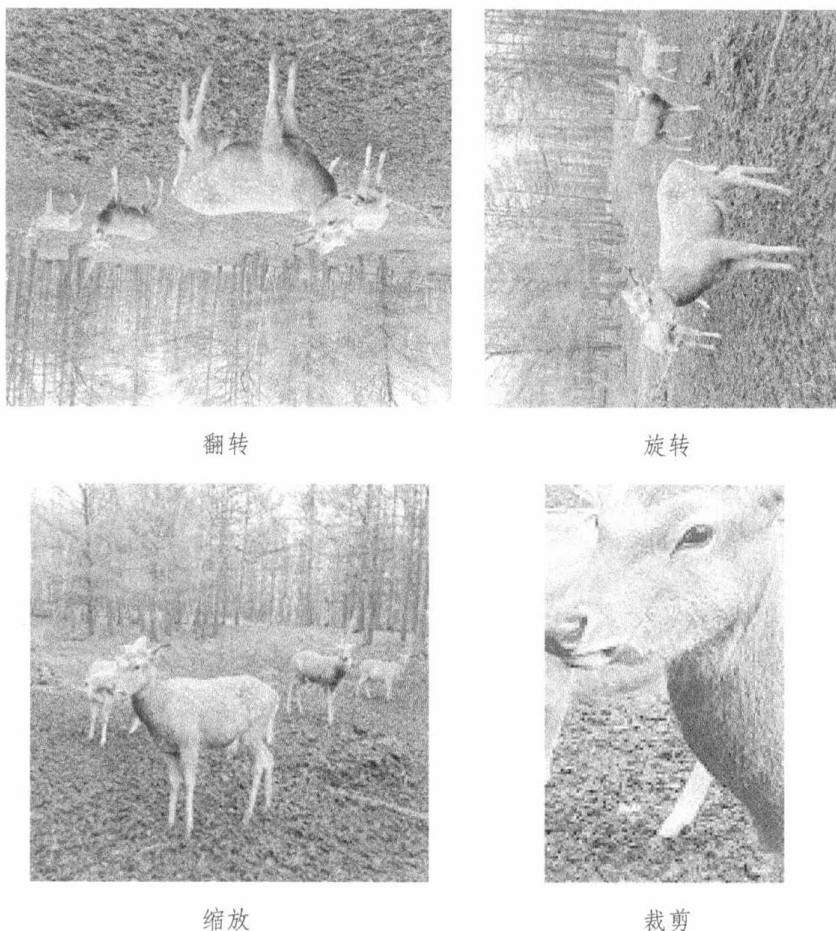


图 7-21 图像翻转 / 旋转 / 缩放 / 裁剪结果

7.3.5 二维码检测和解码

OpenCV C/C++ 可实现二维码的检测与解码。在 OpenCV 头文件中使用命令 `grep -r -w detectAndDecode . -n`，查找如下：

```
opencv-4.9.0/install$ grep -r -w detectAndDecode . -n
```

```
./include/opencv2/objdetect/graphical_code_detector.hpp:45: CV_WRAP std::string
detectAndDecode ( InputArray img, OutputArray points = noArray ( ),
```

```
./include/opencv2/wechat_qrcode.hpp:44: * To simplify the usage, there is a only API:
detectAndDecode
```

```
./include/opencv2/wechat_qrcode.hpp:51: CV_WRAP std::vector<std::string>
detectAndDecode ( InputArray img, OutputArrayOfArrays points = noArray ( ) );
```

可以发现 OpenCV 中支持二维码检测和解码功能的头文件有 `graphical_code_detector`。

hpp 和 wechat_qrcode.hpp。第一个头文件，其中定义了一个类 GraphicalCodeDetector，detectAndDecode 是类的一个方法，如图 7-22 所示。

```

39  /** @brief Both detects and decodes graphical code
40
41  @param img grayscale or color (BGR) image containing graphical code.
42  @param points optional output array of vertices of the found graphical code quadrangle,
    will be empty if not found.
43  @param straight_code The optional output image containing binarized code
44  */
45  CV_WRAP std::string detectAndDecode(InputArray img, OutputArray points = noArray(),
46                                     OutputArray straight_code = noArray()) const;

```

图 7-22 类 GraphicalCodeDetector 和类的方法 detectAndDecode^①

文件 ./include/opencv2/objdetect.hpp:770:class CV_EXPORTS_W_SIMPLE QRCodeDetector: public GraphicalCodeDetector 中的类 QRCodeDetector 继承 GraphicalCodeDetector，于是类 QRCodeDetector 继承了方法 detectAndDecode，实现二维码检测和解码的代码如下：

```
QRCodeDetector QRdetector;
```

```
string info = QRdetector.detectAndDecode ( img, points, straight_qrcode );
```

如图 7-23 所示，实现代码在测试图片中检测到的二维码的四个顶点坐标为 (178, 27)、(358, 163)、(229, 350)、(41, 209)。straight_qrcode 为输出的先矫正后二值化处理的二维码图像。测试图片的尺寸 width 为 429 个像素，height 为 377 个像素，在 RV1126 平台时耗 263 ms。

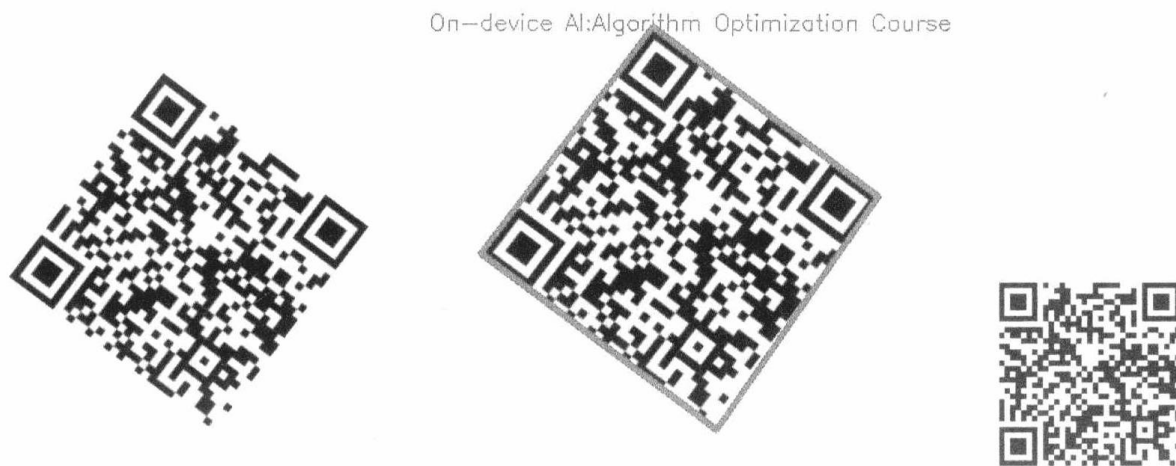


图 7-23 二维码检测、解码和矫正结果（右图为矫正结果）

完整的测试代码如下：

```
#include <stdlib.h>
```

```
#include <iostream>
```

① 内容来源：./include/opencv2/objdetect/graphical_code_detector.hpp:45

```

#include<opencv2/opencv.hpp>
#include <sys/time.h>

using namespace std;
using namespace cv;

int main ( )
{
    Mat img = imread ( " qrcode.png" );
    Mat straight_qrcode;
    vector<Point> points;
    //std::vector<cv::Point> points;

    QRCodeDetector QRdetector;
    struct timeval time_begin, time_end;
    // #include <sys/time.h>
    gettimeofday ( &time_begin, NULL );
    string info = QRdetector.detectAndDecode ( img, points, straight_qrcode );
    gettimeofday ( &time_end, NULL );
    int time = ( 1000000 * ( time_end.tv_sec - time_begin.tv_sec )
        + ( time_end.tv_usec - time_begin.tv_usec ) ) / 1000;
    printf ( "cost: %d ms\n", time )

    Point point1 = Point ( 0, 30 );
    putText ( img, info, point1, FONT_HERSHEY_SIMPLEX, 0.6, Scalar ( 0,
        0, 255 ), 1, 8 );
    for ( int j = 0; j < points.size ( ); j++ )
    {
        printf ( " points %d, %d\n" , points [ j ].x, points [ j ].y );
        if ( j == 3 )
            line ( img, points [ j ], points [ 0 ],
Scalar ( 0, 0, 255 ), 3 );
        else
            line ( img, points [ j ], points [ j + 1 ],
Scalar ( 0, 0, 255 ), 3 );
    }
}

```

```
}  
  
cv::imwrite ( " img.jpg" , img );  
cv::imwrite ( " straight_qrcode.jpg" , straight_qrcode );  
return 0;  
}
```


第 8 章

Neon 指令集加速算法和算子

底层指令加速

Neon 指令是 ARM 处理器的 SIMD 指令集，类似 X86 的 SSE2，它适用于 ARM 中 Cortex-A 和 Cortex-R 系列处理器。Neon Intrinsics 是官方提供的 Neon C 和 C++ 内联函数，可实现底层 Neon 汇编的调用，相比较直接使用汇编更加容易编写和维护。Neon 内联函数接口和数据类型定义在 arm_neon.h 头文件中。本章将讨论如何使用 Neon 内联函数进行 Neon 编程优化算法。

8.1 Neon 寄存器和数据类型

Neon 寄存器，也叫向量寄存器或向量，它包括 16 个 128 bit 四字寄存器 Q0—Q15 或 32 个 64 bit 双字寄存器 D0—D31，其中一个字为 32 bit。

Q0—Q15 中每一个寄存器映射到一对 D 寄存器，比如 Q0 映射 D0 和 D1 寄存器，Q1 映射 D2 和 D3 寄存器，Q2 映射 D4 和 D5 寄存器，以此类推，如图 8-1 所示。

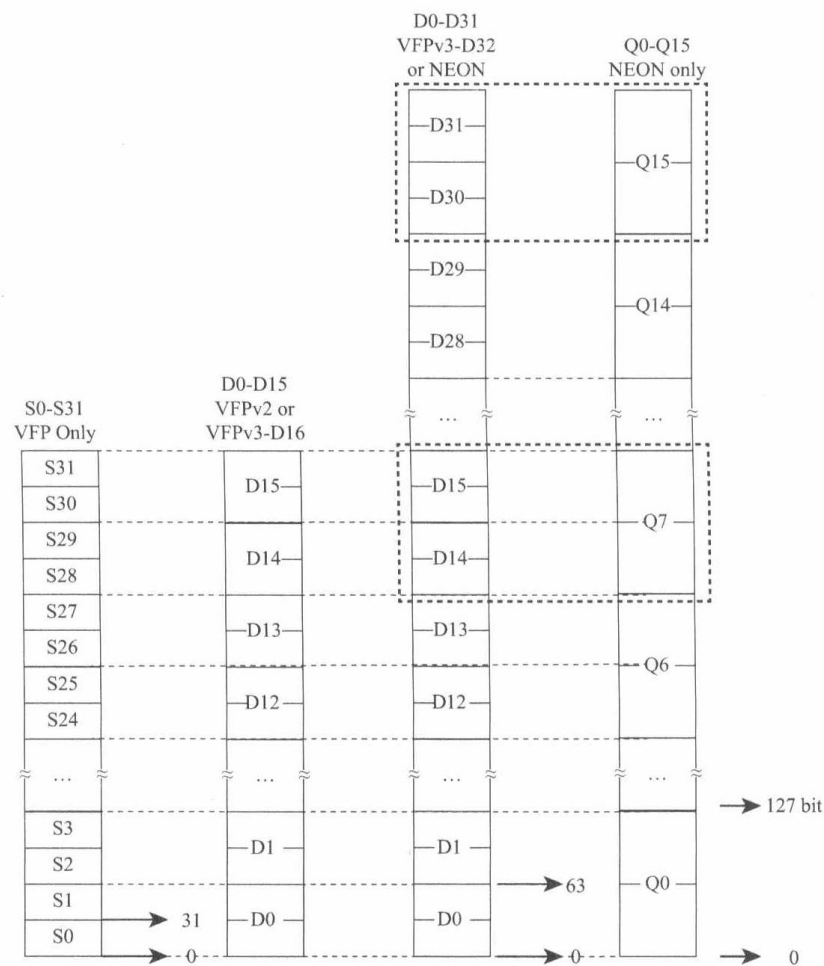


图 8-1 Neon 寄存器^①

① 资料来源：<https://developer.arm.com/documentation/102474/latest> <https://developer.arm.com/documentation/den0018/latest>

图来源：NEON Programmer's Guide Version: 1.0

ARM 处理器的基本数据类型有整数、浮点数、poly，具体如表 8-1 所示。

表 8-1 ARM 基本数据类型

整数	int8、int16、int32、int64
	uint8、uint16、uint32、uint64
浮点数	float16、float32
多项式	poly8、poly16

ARM 处理器中的 Neon 数据类型如表 8-2 所示。

表 8-2 Neon 数据类型

	8 bit	16 bit	32 bit	64 bit
无符号整型 (Unsigned Integer)	U8	U16	U32	U64
	$[0, 2^8)$	$[0, 2^{16})$	$[0, 2^{32})$	$[0, 2^{64})$
有符号整型 (Signed Integer)	S8	S16	S32	S64
	$[-2^7, 2^7)$	$[-2^{15}, 2^{15})$	$[-2^{31}, 2^{31})$	$[-2^{63}, 2^{63})$
未指定类型的整数 (Integer of Unspecified Type)	I8	I16	I32	I64
浮点型 (Floating-point Number)	not available	F16	F32 or F	not available
$\{0, 1\}$ 上的多项式 (Polynomial over $\{0, 1\}$)	P8	P16	not available	not available

Neon 数据类型和基本数据类型的关系是前者是向量，后者是向量元素的基本类型。如图 8-2 所示为 Neon 内联函数中的数据类型的命名规则。

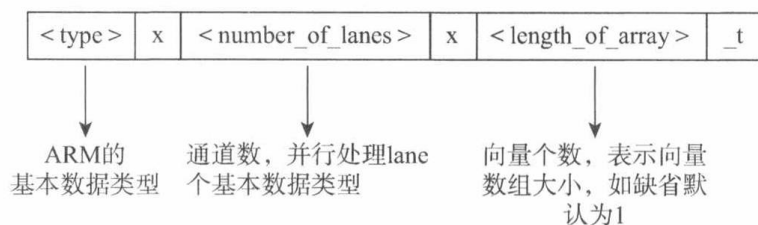


图 8-2 向量数据类型命名规则

当 `length_of_array` 缺省时，默认为 1，各种数据类型命名示例如图 8-3 所示（`arm_neon.h` 文件）：

```

typedef __simd64_int8_t int8x8_t;
typedef __simd64_int16_t int16x4_t;
typedef __simd64_int32_t int32x2_t;
typedef __builtin_neon_di int64x1_t;
#if defined (__ARM_FP16_FORMAT_IEEE) || defined (__ARM_FP16_FORMAT_ALTERNATIVE)
typedef __fp16 float16_t;
typedef __simd64_float16_t float16x4_t;
#endif
typedef __simd64_float32_t float32x2_t;
typedef __simd64_poly8_t poly8x8_t;
typedef __simd64_poly16_t poly16x4_t;
#pragma GCC push_options
#pragma GCC target ("fpu=crypto-neon-fp-armv8")
typedef __builtin_neon_poly64 poly64x1_t;
#pragma GCC pop_options
typedef __simd64_uint8_t uint8x8_t;
typedef __simd64_uint16_t uint16x4_t;
typedef __simd64_uint32_t uint32x2_t;
typedef __builtin_neon_udi uint64x1_t;

typedef __simd128_int8_t int8x16_t;
typedef __simd128_int16_t int16x8_t;
typedef __simd128_int32_t int32x4_t;
typedef __simd128_int64_t int64x2_t;
#if defined (__ARM_FP16_FORMAT_IEEE) || defined (__ARM_FP16_FORMAT_ALTERNATIVE)
typedef __simd128_float16_t float16x8_t;
#endif
typedef __simd128_float32_t float32x4_t;
typedef __simd128_poly8_t poly8x16_t;
typedef __simd128_poly16_t poly16x8_t;
#pragma GCC push_options
#pragma GCC target ("fpu=crypto-neon-fp-armv8")
typedef __builtin_neon_poly64 poly64x2_t __attribute__((__vector_size__(16)));
#pragma GCC pop_options

typedef __simd128_uint8_t uint8x16_t;
typedef __simd128_uint16_t uint16x8_t;
typedef __simd128_uint32_t uint32x4_t;
typedef __simd128_uint64_t uint64x2_t;

typedef float float32_t;

```

图 8-3 向量数据类型命名示例 (length_of_array 缺省)

当 length_of_array 不缺省，被定义的数据类型也叫向量数组（数组的元素是向量），它实际上是一个结构体，结构体中定义向量，而且每一个向量（vector）对应 1 个 Neon 寄存器。向量中可并行处理的基本数据类型的元素个数叫通道数（lanes），如下文定义的向量数组。

```

typedef struct int8x16x2_t
{
    int8x16_t val [ 2 ];
} int8x16x2_t;

```

其中 int8x16x2_t.val [0] 和 int8x16x2_t.val [1] 为 2 个向量（对应 2 个 Neon 寄存器），每个向量包含 16 个元素，且元素类型为 int8。

8.2 Neon 指令类型

Neon 指令按照操作数类型分为以下几种：

(1) 正常指令 (Normal Instructions)

对任何向量类型进行操作，并生成与操作数向量相同大小和通常相同类型的结果向量。例如 Neon 汇编指令：VADD.I16 Q0, Q1, Q2。

(2) 长指令 (Long Instructions)

如图 8-4 所示，对双字向量操作数执行运算，并生成四字向量结果。所生成的元素通常是操作数元素宽度的两倍，并属于同一类型。例如指令：VADDL.S16 Q0, D2, D3。

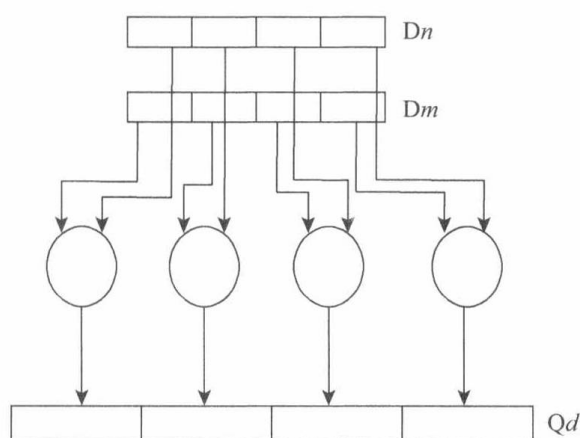


图 8-4 Neon 长指令（操作数 D_n 和 D_m 均是双字向量，生成的结果 Q_d 是四字向量）^①

(3) 宽指令 (Wide Instructions)

如图 8-5 所示，一个双字向量操作数和一个四字向量操作数执行运算，生成四字向量结果。例如指令：VADDW.I16 Q0, Q1, D4。

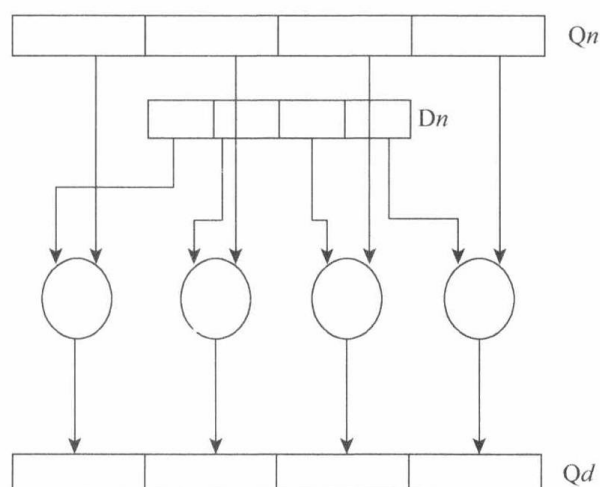


图 8-5 Neon 宽指令（ D_n 是双字向量操作数和 Q_n 是四字向量操作数，生成结果 Q_d 是四字向量）

^① 图 8-4 ~ 图 8-6 来源：NEON Programmer's Guide Version: 1.0

（4）窄指令（Narrow Instructions）

如图 8-6 所示，对四字向量操作数执行运算，并生成双字向量结果。所生成的元素通常是操作数元素宽度的一半。例如指令：VADDHN.I16 D0, Q1, Q2。

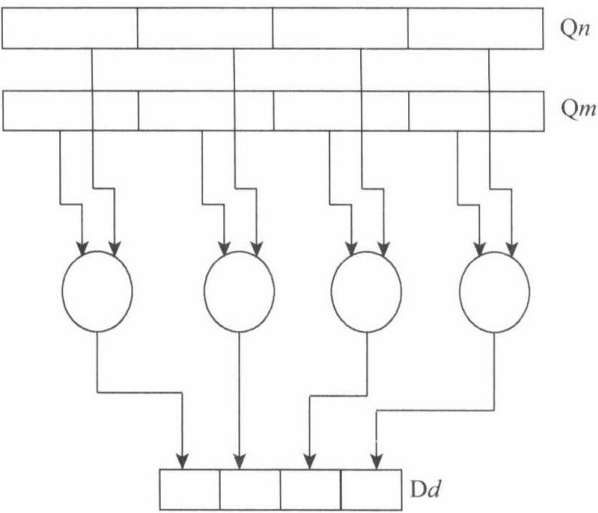


图 8-6 Neon 窄指令（Qn 和 Qm 是四字向量操作数，生成结果 Dd 是双字向量）

（5）饱和指令（Saturating Instructions）

在执行运算中，当向量中某个元素的大小超过该数据类型的可表示范围，则会自动限定在该数据类型的表示范围内，防止溢出。例如指令：VQADD、VQSUB、VQABS、VQSHL、VQRSHL。

8.3 Neon 编程方式和内联函数

Neon 编程方式包括 Neon 汇编指令（Neon Assembler）、编译器自动向量化（Auto-vectorization）、Neon 第三方库（Neon-enabled Libraries）以及 Neon 内联函数（Neon Intrinsics）。

8.3.1 Neon 汇编指令

使用 Neon 汇编指令编程，可直接自主分配寄存器，进而能够发挥硬件平台的最大性能，但是代码开发难度大，工程代码可读性低，平台移植性低。鉴于本书篇幅的限制，本书中关于 Neon 汇编指令不做进一步的讨论。

8.3.2 编译器自动向量化

支持自动向量化的编译器工具有 Arm Compiler 6、Arm C/C++ Compiler、LLVM-clang 和 GCC。GCC 工具实现自动向量化的配置参数包括以下所列：

-ftree-vectorize (需要配置 -O1 及以上才能够生效);
 -mfpu = neon (当配置 -O3 参数时 -ftree-vectorize 默认启动);
 -mcpu (-mcpu 参数选项配置如表 8-3 所示)。

表 8-3 -mcpu 参数选项

CPU	option
Cortex-A5	-mcpu = cortex-a5
Cortex-A7	-mcpu = cortex-a7
Cortex-A8	-mcpu = cortex-a8
Cortex-A9	-mcpu = cortex-a9
Cortex-A15	-mcpu = cortex-a15

根据以上参数选项, 在 RV1126 平台中使用以下编译命令实现编译自动向量化:

```
arm-linux-gnueabi-g++ vadd.cpp -mfpu = neon -mcpu = cortex-a7 -ftree-vectorize -O2 -o test_neon
```

8.3.3 Neon 第三方库

支持 Neon 的第三方库有 CMSIS-NN、ARM Compute Libray、Ne10、Skia、Libyuv 等。

(1) CMSIS-NN

CMSIS (Cortex Microcontroller Software Interface Standard, Cortex 微控制器软件接口标准) 是 ARM 专门针对 Cortex-M 系列提出的一套标准, 它是与供应商无关的硬件抽象层。CMSIS 工程代码地址为 https://github.com/ARM-software/CMSIS_5.git, 表 8-4 所示为 CMSIS 的代码结构, 其中 NN 是 CMSIS 中一个组件, 用于支持 ARM Cortex-M 系列处理器内核进行神经网络计算。NN 分为 NNFunctions 和 NNSupportFunctions 两个部分, 其中 NNFunctions 包含卷积函数、深度可分离卷积函数、全连接函数、池化函数和激活函数等神经网络中常用的函数, NNSupportFunctions 函数实现关键函数优化, 它包括 NNFunctions 中使用的数据转换 (用定点 8 或 16 bits 替代浮点计算) 和激活功能表 (通过查表实现激活函数计算)。

表 8-4 CMSIS 结构^①

CMSIS	目标处理器	描述
Core (M)	All Cortex-M, SecurCore	Cortex-M 处理器核心和外围设备的标准化 API。包括 Cortex-M4/M7/M33/M35P SIMD 指令的固有功能
Core (A)	Cortex-A5/A7/A9	Cortex-A5/A7/A9 处理器核心和外围设备的标准化 API 和基本运行时系统

① 摘自: https://arm-software.github.io/CMSIS_5/General/html/index.html

续表

CMSIS	目标处理器	描述
Driver	All Cortex	中间件的通用外围驱动程序接口。将微控制器外围设备与实现例如通信堆栈、文件系统或图形用户界面的中间件连接
DSP	All Cortex-M	DSP 库集合，具有 60 多种功能，适用于各种数据类型：定点（分数 q7、q15、q31）和单精度浮点（32 位）。针对 SIMD 指令集优化的实现可用于 Cortex-M4/M7/M33/M35P
NN	All Cortex-M	收集高效的神经网络内核，以最大限度地提高 Cortex-M 处理器内核的性能和减少内存占用
RTOS v1	Cortex-M0/M0+/M3/M4/M7	实时操作系统的通用 API 以及基于 RTX 的参考实现。它支持可以跨多个 RTOS 系统工作的软件组件
RTOS v2	All Cortex-M, Cortex-A5/A7/A9	使用 Armv8-M 支持、动态对象创建、多核系统规定、二进制兼容接口扩展 CMSIS-RTOS v1
Pack	All Cortex-M, SecurCore, Cortex-A5/A7/A9	描述软件组件、设备参数和评估板支持的传递机制。它简化了软件重用和产品生命周期管理（PLM）
Build	All Cortex-M, SecurCore, Cortex-A5/A7/A9	一组提高生产率的工具、软件框架和 workflows，例如通过持续集成（CI）
SVD	All Cortex-M, SecurCore	可用于在调试器或 CMSIS 核心头文件中创建外设感知的设备的外设描述
DAP	All Cortex	与 CoreSight 调试访问端口接口的调试单元的固件
Zone	All Cortex-M	定义描述系统资源并将这些资源划分为多个项目和执行区域的方法

NN 的框架如图 8-7 所示，从图中可以很好地理解 NNFunctions 和 NNSupportFunctions 的关系。

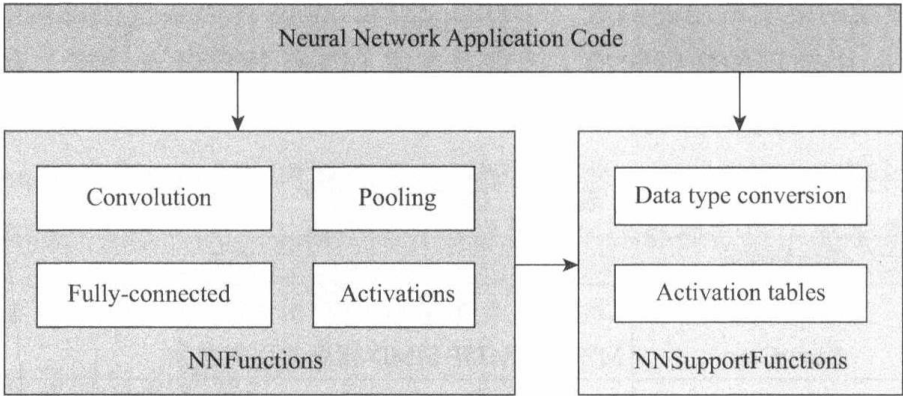


图 8-7 NN 框架^①

① 图来源：<https://www.keil.com/pack/doc/CMSIS/NN/html/index.html>

(2) ARM Compute Libray

ARM Compute Libray (ACL) 库的结构如图 8-8 所示, Neon 部分实现 Cortex-A CPU 架构的性能优化, OpenCL 部分实现 Mali GPU 架构的性能优化^①。

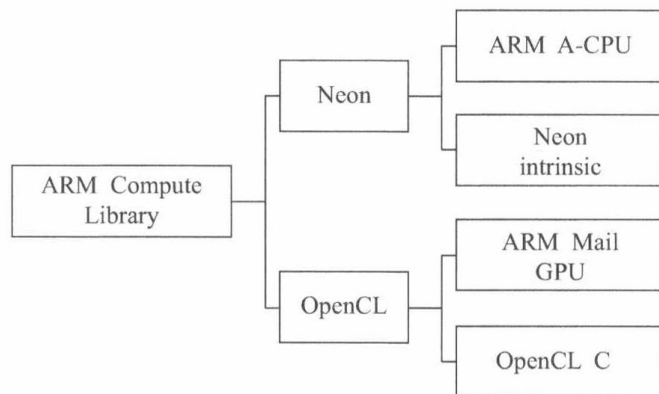


图 8-8 ARM Compute Library

Neon 优化的高性能接口实现 GEMM、Winograd、FFT、滑窗 CNN 和单精度矩阵乘法 SGEMM^② 等。

ARM Compute Libray 库的编译方法如下:

```
ComputeLibrary$ scons Werror=0 debug=0 asserts=0 neon=1 opengl=1 os=linux
arch=armv7a
```

.....

```
arm-linux-gnueabi-hf-ranlib build/tests/framework/libarm_compute_test_framework.a
```

```
scons: done building targets.
```

(3) Ne10

Ne10 是 ARM 官方发布的支持浮点运算、矢量计算、矩阵操作的一个开源库, 代码仓库地址为 <https://github.com/projectNe10/Ne10>。其主要模块包括信号处理 DSP (FFT 快速傅立叶变换、FIR/IIR 滤波等)、数学 math (矢量、矩阵运算等)、图像处理 imagproc (图像缩放、旋转处理) 等。以下为 Ne10 的具体模块列表^③:

Math Functions

Vector Addition

Matrix Addition

Vector Subtraction ($w[i] = u[i] - v$)

① 代码仓库地址: <https://github.com/ARM-software/ComputeLibrary>

② 参考资料: https://github.com/ARM-software/ComputeLibrary/blob/master/arm_compute/runtime/NEON/NEFunctions.h

③ 来源: <http://projectne10.github.io/Ne10/doc/modules.html>

Vector Subtraction From ($w[i] = v - u[i]$)

Matrix Subtraction

Vector Multiplication

Vector Multiply-Accumulate

Matrix Multiplication

Matrix-Vector Multiplication

Vector Division

Vector Assignment

Vector Modulus (Length)

Vector Normalization

Vector Element-wise Absolute Value

Vector Dot Product

Vector Cross Product

Matrix Determinant Calculation

Matrix Inversion

Matrix Transposition

Matrix Identity Assignment

Signal Processing Functions

Complex-to-Complex FFT (Floating & Fixed Point)

Real-to-Complex FFT (Floating & Fixed Point)

Finite Impulse Response (FIR) Filters

Finite Impulse Response (FIR) Decimation

Finite Impulse Response (FIR) Interpolation

Finite Impulse Response (FIR) Lattice Filters

Finite Impulse Response (FIR) Sparse Filters

Infinite Impulse Response (IIR) Lattice Filters

Image Processing Functions

Image Box Filter (Blur)

Image Resize

Image Rotation

Physics Functions

Collision Detection

(4) Skia

Skia 是开源的 2D 图形库，可用于 web 浏览器和操作系统的图形引擎。

(5) Libyuv

Libyuv 是实现 YUV 缩放和转换等功能的开源库。它的代码仓库地址为：

<https://developer.arm.com/Architectures/Neon>。

8.3.4 Neon 内联函数

Neon 内联函数的命名规则如下：

`v <mod> <opname> <shape> <flags>_<type>`，其中 `opname` 代表指令名，`type` 代表基本数据类型简写。

(1) `mod` 前缀修饰符表示计算方式，选项有 `q`、`h`、`d`、`p` 和 `r`

`mod=q` 表示饱和计算，如内联函数 `int8x8_t vqadd_s8 (int8x8_t N, int8x8_t M)`，用于实现 `N` 和 `M` 两个向量的对应元素相加，如果结果溢出，就做饱和处理，其完整测试代码如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <arm_neon.h>
int main ( )
{
    int8_t vector_0 [ 8 ] = {127, 127, 127, -128, 10, 10, -10, -128};
    int8_t vector_1 [ 8 ] = {127, 127, 10, -128, 20, 100, -100, -2};
    int8_t result [ 8 ];

    int8x8_t vec0 = vld1_s8 ( vector_0 );
    int8x8_t vec1 = vld1_s8 ( vector_1 );
    int8x8_t vec_r = vqadd_s8 ( vec0, vec1 );
    vst1_s8 ( result, vec_r );
    // 打印结果
    for ( int i = 0; i < 8; i++ ) {
        printf ( " %d " , result [ i ] );
    }
    printf ( " \n" );
}
```

编译方法：`arm-linux-gnueabi-g++ vqadd_s8-neon.cpp -o test_neon`

完整的测试结果见表 8-5。

表 8-5 mod=q 测试结果

操作数	执行结果
vec0	127, 127, 127, -128, 10, 10, -10, -128
vec1	127, 127, 10, -128, 20, 100, -100, -2
vadd_s8	-2 -2 -119 0 30 110 -110 126
	计算机系统用补码；正数的补码等于原码；负数使用补码表示，即绝对值按位取反后再加 1 127+127 $\begin{aligned} & (01111111)_b \\ & + (01111111)_b \\ & = (11111110)_b \\ & = (-2)_d \end{aligned}$ 127+10 $\begin{aligned} & (01111111)_b \\ & + (00001010)_b \\ & = (10001001)_b \\ & = (-119)_d \end{aligned}$ -128-128 $\begin{aligned} & (10000000)_b \\ & + (10000000)_b \\ & = (00000000)_b \\ & = (0)_d \end{aligned}$
	-128-2 $\begin{aligned} & (10000000)_b \\ & + (11111110)_b \\ & = (01111110)_b \\ & = (126)_d \end{aligned}$
vqadd_s8	127 127 127 -128 30 110 -110 -128
vec0	12, 12, 12, -12, 10, 10, -10, -12
vec1	12, 12, 10, -12, 20, 100, -100, -2
vadd_s8	24 24 22 -24 30 110 -110 -14
vqadd_s8	24 24 22 -24 30 110 -110 -14

mod=h 表示折半计算，如内联函数 `int8x8_t vhadd_s8 (int8x8_t N, int8x8_t M)`，用于实现 N 和 M 两个向量的对应元素相加，然后右移一位（除以 2）。其完整的测试结果见表 8-6。

表 8-6 mod=h 测试结果

操作数	执行结果
vec0	127, 127, 127, -128, 10, 10, -10, -128
vec1	127, 127, 10, -128, 20, 100, -100, -2
vadd_s8	-2 -2 -119 0 -56 -56 -56 126
vhadd_s8	127 127 68 -128 15 55 -55 -65
vec0	12, 12, 12, -12, 10, 10, -10, -12
vec1	12, 12, 10, -12, 20, 100, -100, -2
vadd_s8	24 24 22 -24 30 110 -110 -14
vhadd_s8	12 12 11 -12 15 55 -55 -7

mod=r 表示舍入计算，如内联函数 `int8x8_t vrhadd_s8 (int8x8_t N, int8x8_t M)`，用于实现 N 和 M 两个向量的对应元素相加，然后右移一位（除以 2），最终需要将得到的结果做 rounding（舍入）运算。其完整的测试结果见表 8-7。

表 8-7 mod=r 测试结果

操作数	执行结果
vec0	127, 127, 127, -128, 10, 10, -10, -128
vec1	127, 127, 10, -128, 20, 100, -100, -2
vhadd_s8	127 127 68 -128 15 55 -55 -65
vrhadd_s8	127 127 69 -128 15 55 -55 -65
vec0	12, 12, 12, -12, 10, 10, -10, -12
vec1	12, 12, 10, -12, 20, 100, -100, -2
vhadd_s8	12 12 11 -12 15 55 -55 -7
vrhadd_s8	12 12 11 -12 15 55 -55 -7

mod=d 表示加倍计算，如内联函数 `int32x4_t vqdmull_s16 (int16x4_t N, int16x4_t M)`，用于实现 N 和 M 两个向量的对应元素相乘，再乘以 2，如果结果溢出，就做饱和处理，其完整测试代码如下：


```

#include <stdio.h>
#include <stdlib.h>
#include <arm_neon.h>
#include <math.h>

int main ( )
{
    int16_t vector_0 [ 8 ] = {1, 2, 32767, -32768, 1, 0, 7, 8};
    int16_t vector_1 [ 8 ] = {11, 12, 32767, -32768, 2, 16, 17, 18};
    int32_t result [ 8 ];

    int16x4_t vec0 = vld1_s16 ( vector_0 );
    int16x4_t vec1 = vld1_s16 ( vector_1 );

    int32x4_t vec_r = vqdmull_s16 ( vec0, vec1 );

    vst1q_s32 ( result, vec_r );
    // 打印结果
    for ( int i = 0; i < 8; i++ ) {
        printf ( "%d", result [ i ] );
    }
    printf ( " \n" );
}

```

测试结果：

```
22 48 2147352578 2147483647 22 -1493953296 -1493752456 -1365370164
```

mod=p 表示成对 (pairwise) 计算，如内联函数 `int8x8_t vpadd_s8 (int8x8_t N, int8x8_t M)`，用于实现 N 和 M 两个向量中每个向量的相邻的一对元素相加，将得到的结果作为输出。

又比如内联函数 `int8x8_t vpmin_s8 (int8x8_t N, int8x8_t M)`，用于实现 N 和 M 两个向量中每个向量的相邻的一对元素的求最小值，得到的结果作为输出；同理 `vpmax_s8` 求最大值。

完整的测试代码如下：

```

#include <stdio.h>
#include <stdlib.h>
#include <arm_neon.h>

```

```

int main ( )
{
    int8_t vector_0 [ 8 ] = {12, 12, 12, -12, 10, 10, -10, -12};
    int8_t vector_1 [ 8 ] = {12, 12, 10, -12, 20, 100, -100, -2};
    int8_t result [ 8 ];

    int8x8_t vec0 = vld1_s8 ( vector_0 );
    int8x8_t vec1 = vld1_s8 ( vector_1 );

    int8x8_t vec_r = vpadd_s8 ( vec0, vec1 );
    //int8x8_t vec_r = vpmin_s8 ( vec0, vec1 );
    //int8x8_t vec_r = vpmax_s8 ( vec0, vec1 );
    vst1_s8 ( result, vec_r );

    // 打印结果
    for ( int i = 0; i < 8; i++ ){
        printf ( " %d" , result [ i ] );
    }
    printf ( " \n" );
}

```

完整的测试结果见表 8-8。

表 8-8 mod=p 测试结果

操作数	执行结果
vec0	12, 12, 12, -12, 10, 10, -10, -12
vec1	12, 12, 10, -12, 20, 100, -100, -2
vpadd_s8	24 0 20 -22 24 -2 120 -102
vpmin_s8	12 -12 10 -12 12 -12 20 -100
vpmax_s8	12 12 10 -10 12 10 100 -2

(2) shape 后缀修饰符表示指令类型, 选项有 l (Long)、n (Narrow)、w (Wide) 和 q
 shape=l 表示长指令, 如内联函数 `int64x2_t vaddl_type (int32x2_t N, int32x2_t M);`
 shape=n 表示窄指令, 如内联函数 `int32x4_t vaddl_s16 (int16x4_t N, int16x4_t M);`
 shape=w 表示宽指令, 如内联函数 `int64x2_t vaddw_s32 (int32x2_t N, int32x2_t M);`
 shape=q 表示指令操作 Q 寄存器; 反之 D 寄存器。

(3) flags 表示指令标志, 选项有 high、low、n 和 lane

当 flags=high 或 low 表示对向量的高半部分或低半部分操作, 实现从 Q 寄存器访问 D 寄存器, 如内联函数 Result_t vget_high_type (Vector_t N) 和 Result_t vget_low_type (Vector_t N) 分别返回输入 128 bit 向量的高半部分或低半部分, 得到一个 64 bit 向量, 其元素数量是输入向量的一半。完整测试代码如下:

```
#include <stdio.h>
#include <arm_neon.h>

int main ( )
{
    int8_t vector_0 [ 16 ] = {1, 2, 3, 4, 5, 6, 7, 8, 11, 12, 13, 14, 15, 16,
17, 18};
    int8_t result [ 8 ];

    int8x16_t vec0 = vld1q_s8 ( vector_0 );
    int8x8_t vec_r = vget_high_s8 ( vec0 );
    vst1_s8 ( result, vec_r );

    // 打印结果
    for ( int i = 0; i < 8; i++ ){
        printf ( " %d" , result [ i ] );
    }
    printf ( " \n" );
}
```

代码运行结果为 11 12 13 14 15 16 17 18。

测试代码中 int8x8_t vec_r = vget_high_s8 (vec0) 改为 int8x8_t vec_r = vget_low_s8 (vec0), 代码运行结果为 1 2 3 4 5 6 7 8。

当 flags=n 表示有标量参与向量的计算, 如内联函数 int32x4_t vmulq_n_s32 (int32x4_t N, int32_t M), 用于实现标量 M 乘以向量 N。示例代码如下:

```
#include <stdio.h>
#include <arm_neon.h>

int main ( )
{
    int32_t vector_0 [ 8 ] = {1, 2, 3, 4, 5, 6, 7, 8};
```

```

int32_t result [ 8 ];
int32_t M = 100;

int32x4_t vec0 = vld1q_s32 ( vector_0 );
int32x4_t vec_r = vmulq_n_s32 ( vec0, M );
vst1q_s32 ( result, vec_r );

// 打印结果
for ( int i = 0; i < 8; i++ ) {
    printf ( " %d" , result [ i ] );
}
printf ( " \n" );
}

```

代码运行结果为 100 200 300 400 22 -1493777168 -1493576328 -1359467828。

当 flags=lane 表示向量中某个通道参与向量的计算，如内联函数 float32_t vgetq_lane_f32 (float32x4_t N, int n)，用于实现读取向量 N 中第 n 个通道的值。示例代码如下：

```

#include <stdio.h>
#include <arm_neon.h>

int main ( )
{
    float vector_0 [ 8 ] = { 1., 2., 3., 4., 5., 6., 7., 8. };
    //int n = 3;

    float32x4_t vec0 = vld1q_f32 ( vector_0 );
    float r = vgetq_lane_f32 ( vec0, 3 );

    // 打印结果
    printf ( " %f" , r );
    printf ( " \n" );
}

```

代码运行结果为 4.000000

编写代码需要注意：vgetq_lane_f32 内联函数中第二个参数需要是常数，如果传入的是变量，编译会报以下错误：

In function ' float32_t vgetq_lane_f32 (float32x4_t, int) ' ,

```

    inlined from 'int main ( ) ' at vmulq_n_type-neon.cpp:28:29:
/opt/atk-dlrv1126-toolchain/prebuilts/gcc/linux-x86/arm/gcc-arm-8.3-2019.03-x86_64-
arm-linux-gnueabi/lib/gcc/arm-linux-gnueabi/8.3.0/include/arm_neon.h:6267:50:
error: argument 2 must be a constant immediate
    return ( float32_t ) __builtin_neon_vget_lanev4sf ( __a, __b );

```

8.4 Neon 常用内联函数介绍

Neon 指令按照功能可分为以下：

- ① 类型转换指令（Intrinsics type conversion）
 - ② 加载存储指令（Load and store）
 - ③ 算术运算指令（Arithmetic）
 - ④ 数据处理指令（Data processing）
 - ⑤ 向量乘法指令（Multiply）
 - ⑥ 逻辑和比较运算指令（Logical and compare）
- 包括与、或、非、等于、大于、小于等运算。
- ⑦ 浮点指令（Floating-point）
 - ⑧ 移位指令（Shift）
 - ⑨ 置换指令（Permutation）
 - ⑩ 其他指令（Miscellaneous）^①

8.4.1 类型转换指令

类型转换指令（Intrinsics type conversion）用于实现数据类型的转换。

1. vreinterpret

vreinterpret 指令用于对数据的强制类型转换，其内联函数定义如下：

```
Result_t vreinterpret_DSTtype_SRCtype ( Vector1_t N );
```

```
Result_t vreinterpretq_DSTtype_SRCtype ( Vector1_t N );
```

其中符号 SRCtype、DSTtype、Vector1_t、Result_t 的数据类型选项如表 8-9 所示：

① 多写和测试验证 Neon 代码，并且思考如何用 Neon 去优化算法。慢慢就找到感觉，能掌握如何使用指令实现高性能计算。

表 8-9 vreinterpret 指令数据类型选项

	SRCTYPE 或 DSTTYPE	Vector1_t 或 Result_t
D0—D31 寄存器 数据宽度为 64 bit	s8	int8x8_t
	s16	int16x4_t
	s32	int32x2_t
	s64	int64x1_t
	u8	uint8x8_t
	u16	uint16x4_t
	u32	uint32x2_t
	u64	uint64x1_t
	f16	float16x4_t
	f32	float32x2_t
	p8	poly8x8_t
	p16	poly16x4_t
Q0—Q15 寄存器 数据宽度为 128 bit	s8	int8x16_t
	s16	int16x8_t
	s32	int32x4_t
	s64	int64x2_t
	u8	uint8x16_t
	u16	uint16x8_t
	u32	uint32x4_t
	u64	uint64x2_t
	f16	float16x8_t
	f32	float32x4_t
	p8	poly8x16_t
	p16	poly16x8_t
数据类型定义: <type> <size>x<number_of_lanes>x<length_of_array>_t		

如内联函数 `uint32x2_t vreinterpret_u32_s32 (int32x2_t N)` 实现将有符号的 32 位整数 (s32) 转换为无符号的 32 位整数 (u32);

如内联函数 `uint32x4_t vreinterpret_u32_f32 (float32x4_t N)` 实现将有符号的 32 位浮点数 (f32) 转换为无符号的 32 位整数 (u32)。

(2) vcombine

vcombine 指令用于将两个 64 bit 向量合并成一个 128 bit 向量, 当然输出向量的元素数量是每个输入向量元素数量的两倍。其内联函数定义如下:

`Result_t vcombine_type ( Vector1_t N, Vector2_t M );`
具体的参数选项如表 8-10 所示。

表 8-10 内联函数参数选项

Result_t	type	Vector1_t	Vector2_t
int8x16_t	s8	int8x8_t	int8x8_t
int16x8_t	s16	int16x4_t	int16x4_t
int32x4_t	s32	int32x2_t	int32x2_t
int64x2_t	s64	int64x1_t	int64x1_t
uint8x16_t	u8	uint8x8_t	uint8x8_t
uint16x8_t	u16	uint16x4_t	uint16x4_t
uint32x4_t	u32	uint32x2_t	uint32x2_t
uint64x2_t	u64	uint64x1_t	uint64x1_t
float16x8_t	f16	float16x4_t	float16x4_t
float32x4_t	f32	float32x2_t	float32x2_t
poly8x16_t	p8	poly8x8_t	poly8x8_t
poly16x8_t	p16	poly16x4_t	poly16x4_t

8.4.2 加载存储指令

加载指令（Load）实现从内存中加载数据到 Neon 寄存器中。存储指令（Store）实现存储 Neon 寄存器里的数据到内存中。Load/Store 指令操作如图 8-9 所示。

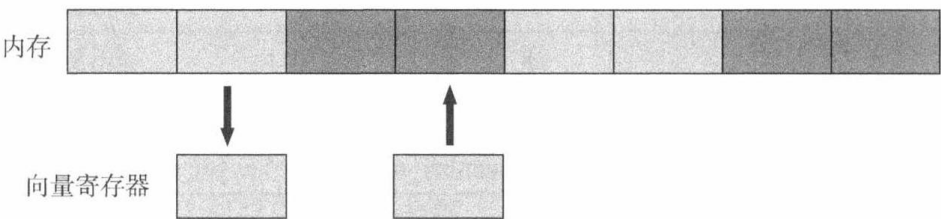


图 8-9 Load/Store 指令操作示意图

(1) vld1 和 vst1

vld1 指令用于从内存加载数据到 1 个向量，其内联函数定义如下：

```
Result_t vld1_type ( Scalar_t* N );  
Result_t vld1q_type ( Scalar_t* N );
```

以上表达式中后缀不带 q，表示对应 64 bit 的向量 D 寄存器；带有 q 后缀，表示对

应 128 bit 的向量 Q 寄存器。下文介绍的其他指令中后缀 q 同理。

其中符号 Result_t、type、Scalar_t 的数据类型选项如表 8-11 所示：

表 8-11 vld1 指令数据类型选项

D ₀ —D ₃₁ 寄存器 64 bit		
Result_t	type	Scalar_t
int8x8_t	s8	int8_t
int16x4_t	s16	int16_t
int32x2_t	s32	int32_t
int64x1_t	s64	int64_t
uint8x8_t	u8	uint8_t
uint16x4_t	u16	uint16_t
uint32x2_t	u32	uint32_t
uint64x1_t	u64	uint64_t
float16x4_t	f16	float16_t
float32x2_t	f32	float32_t
poly8x8_t	p8	poly8_t
poly16x4_t	p16	poly16_t
Q ₀ —Q ₁₅ 寄存器 128 bit		
Result_t	type	Scalar_t
int8x16_t	s8	int8_t
int16x8_t	s16	int16_t
int32x4_t	s32	int32_t
int64x2_t	s64	int64_t
uint8x16_t	u8	uint8_t
uint16x8_t	u16	uint16_t
uint32x4_t	u32	uint32_t
uint64x2_t	u64	uint64_t
float16x8_t	f16	float16_t
float32x4_t	f32	float32_t
poly8x16_t	p8	poly8_t
poly16x8_t	p16	poly16_t

如内联函数 `uint32x4_t vld1q_f32 ( uint32_t const *ptr )` 实现以 `ptr` 为起始地址加载 4 个 `float32` 类型的数据到 Neon 向量寄存器里，其完整的示例代码如下。

```
#include <stdio.h>
#include <stdlib.h>
#include <arm_neon.h>①
int main ( )
{
    float src [ 12 ] = {0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 0.10, 0.11,
0.12};
    float dst [ 12 ] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
    float32x4_t vec0 = vld1q_f32 ( &src [ 1 ] );
    vst1q_f32 ( dst, vec0 );
    for ( int i = 0; i < 12; i++ ) {
        printf ( " %f" , dst [ i ] );
    }
    printf ( " \n" );
}
```

代码分析：

函数 `float32x4_t vec0 = vld1q_f32 ( &src [ 1 ] )` 实现一次加载以 `src [ 1 ]` 地址起始的 4 个 `float32` 类型的数据到向量 `vec0` 中。函数 `vst1q_f32 ( dst, vec0 )` 实现将向量 `vec0` 中 4 个 `float32` 类型的数据赋值给以 `dst` 为起始地址的 4 个 `float32` 类型的数据。

编译代码：

```
( base ) user@user: /neon$ arm-linux-gnueabi-g++ test_neon.cpp -g -o test_neon
```

```
( base ) user@user: /neon$ adb push test_neon /
```

代码运行结果：

```
[ root@ATK-DLRV1126:/ ] # ./test_neon
0.200000 0.300000 0.400000 0.500000 0.000000 0.000000 0.000000 0.000000 0.000000
0.000000 0.000000 0.000000
```

`vst1` 指令实现存储 1 个向量到内存，其内联函数定义如下：

```
void vst1_type ( Scalar_t* N, Vector_t M );
void vst1q_type ( Scalar_t* N, Vector_t M );
```

① 头文件路径：`/opt/atk-dlrv1126-toolchain/prebuilts/gcc/linux-x86/arm/gcc-arm-8.3-2019.03-x86_64-arm-linux-gnueabi/lib/gcc/arm-linux-gnueabi/8.3.0/include/arm_neon.h`

其中符号 Result_t、type、Scalar_t 的数据类型选项如表 8-12 所示：

表 8-12 vstl 指令数据类型选项

D ₀ —D ₃₁ 寄存器 64 bit		
type	Scalar_t	Vector_t
s8	int8_t	int8x8_t
s16	int16_t	int16x4_t
s32	int32_t	int32x2_t
s64	int64_t	int64x1_t
u8	uint8_t	uint8x8_t
u16	uint16_t	uint16x4_t
u32	uint32_t	uint32x2_t
u64	uint64_t	uint64x1_t
f16	float16_t	float16x4_t
f32	float32_t	float32x2_t
p8	poly8_t	poly8x8_t
p16	poly16_t	poly16x4_t
Q ₀ —Q ₁₅ 寄存器 128 bit		
type	Scalar_t	Vector_t
s8	int8_t	int8x16_t
s16	int16_t	int16x8_t
s32	int32_t	int32x4_t
s64	int64_t	int64x2_t
u8	uint8_t	uint8x16_t
u16	uint16_t	uint16x8_t
u32	uint32_t	uint32x4_t
u64	uint64_t	uint64x2_t
f16	float16_t	float16x8_t
f32	float32_t	float32x4_t
p8	poly8_t	poly8x16_t
p16	poly16_t	poly16x8_t

以具体的内联函数 `vst1q_u32 ( dst, vect )` 为例，其完整测试代码如下：

```
#include <arm_neon.h>
#include <stdio.h>
#include <stdlib.h>

#include <iostream>
using namespace std;

#define LENGTH 10000
int main ( )
{
    int sum = 0;
    uint32_t array [ LENGTH ];
    for ( uint i=0; i<LENGTH; i++ ){
        array [ i ] =6;
    }
    for ( int i=0; i <LENGTH; i++ )
        std::cout << array [ i ] << " , " ; //<< std::endl;
    cout << " \n" << endl;
    uint32x4_t in1, in2;

    uint32_t dst [ LENGTH ];

    //double t_1 = static_cast<double> ( getTickCount ( ) );
    clock_t startTime = clock ( );
    for ( uint i=0; i<LENGTH/4; ++i )
    {
        uint * temp =( array+4*i );
        in1 = vld1q_u32 ( temp );
        in2 = vld1q_u32 ( temp );
        in1 = vaddq_u32 ( in1, in2 );
        vst1q_u32 ( dst+4*i, in1 );
        //sum = vaddvq_u32 ( in1 );
    }
    //t_1 =(( double ) getTickCount ( ) - t_1 ) / getTickFrequency ( );
```

```

clock_t endTime = clock ( );
double totalTime = ( double )( endTime-startTime )/ CLOCKS_PER_SEC; // 秒
cout <<" time cost:" << totalTime <<" s" << endl;

for ( int i=0; i<LENGTH; i++ )
    std::cout << dst [ i ] <<" , " ; //<< std::endl;
cout <<" \n" << endl;

return 0;
}

```

代码分析:

函数 `in1 = vld1q_u32 ( temp )` 实现加载 `temp` 地址起始的 4 个 `uint32_t` 类型的数据到 `in1`, 其中 `in1` 的类型为 `uint32x4_t`。函数 `vst1q_u32 ( dst+4*i, in1 )` 实现将 `in1` 中 4 个 `uint32_t` 类型的数据赋值给起始地址为 `dst+4*i` 空间中。

代码运行结果:

基于 V853 (ARM Cortex-A7 CPU core @ 1 GHz) 平台:

time cost: 0.000865292 s

基于 RV1126 (四核 32 位 ARM Cortex-A7 @ 1.5 GHz) 平台:

time cost: 0.000281 s

(2) `vld2` 和 `vst2`

`vld2` 指令用于从内存加载数据到 2 个向量, 其内联函数定义如下:

`Result_t vld2_type ( Scalar_t* N );`

`Result_t vld2q_type ( Scalar_t* N );`

其中符号 `Result_t`、`type`、`Scalar_t` 的数据类型选项如表 8-13 所示:

表 8-13 `vld2` 指令数据类型选项

D ₀ —D ₃₁ 寄存器 64 bit		
Result_t	type	Scalar_t
<code>int8x8x2_t</code>	<code>s8</code>	<code>int8_t</code>
<code>int16x4x2_t</code>	<code>s16</code>	<code>int16_t</code>
<code>int32x2x2_t</code>	<code>s32</code>	<code>int32_t</code>
<code>int64x1x2_t</code>	<code>s64</code>	<code>int64_t</code>
<code>uint8x8x2_t</code>	<code>u8</code>	<code>uint8_t</code>
<code>uint16x4x2_t</code>	<code>u16</code>	<code>uint16_t</code>

续表

D ₀ —D ₃₁ 寄存器 64 bit		
Result_t	type	Scalar_t
uint32x2x2_t	u32	uint32_t
uint64x1x2_t	u64	uint64_t
float16x4x2_t	f16	float16_t
float32x2x2_t	f32	float32_t
poly8x8x2_t	p8	poly8_t
poly16x4x2_t	p16	poly16_t
Q ₀ —Q ₁₅ 寄存器 128 bit		
Result_t	type	Scalar_t
int8x16x2_t	s8	int8_t
int16x8x2_t	s16	int16_t
int32x4x2_t	s32	int32_t
uint8x16x2_t	u8	uint8_t
uint16x8x2_t	u16	uint16_t
uint32x4x2_t	u32	uint32_t
float16x8x2_t	f16	float16_t
float32x4x2_t	f32	float32_t
poly8x16x2_t	p8	poly8_t
poly16x8x2_t	p16	poly16_t

vst2 指令实现存储 2 个向量到内存，其内联函数定义如下：

```
void vst2_type ( Scalar_t* N, Vector_t M );
```

```
void vst2q_type ( Scalar_t* N, Vector_t M );
```

以具体的内联函数 vld2q_f32 和 vst2q_f32 为例，其完整测试代码如下：

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
#include <arm_neon.h>
```

```
int main ( )
```

```
{
```

```
    float src [ 12 ] = {0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 0.10, 0.11, 0.12};
```



```
float dst [ 12 ] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
float dst_0 [ 12 ] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
float dst_1 [ 12 ] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
float32x4x2_t vec0 = vld2q_f32 ( src );
```

```
vst1q_f32 ( dst_0, vec0.val [ 0 ] ); //float32x4_t
```

```
vst1q_f32 ( dst_1, vec0.val [ 1 ] );
```

```
for ( int i = 0; i < 12; i++ ) {
```

```
    printf ( " %f" , dst_0 [ i ] );
```

```
}
```

```
printf ( " \n" );
```

```
for ( int i = 0; i < 12; i++ ) {
```

```
    printf ( " %f" , dst_1 [ i ] );
```

```
}
```

```
printf ( " \n" );
```

```
vst2q_f32 ( dst, vec0 );
```

```
for ( int i = 0; i < 12; i++ ) {
```

```
    printf ( " %f" , dst [ i ] );
```

```
}
```

```
printf ( " \n" );
```

```
}
```

代码运行结果:

```
[root@ATK-DLRV1126:/] # ./test_neon
```

```
0.100000 0.300000 0.500000 0.700000 0.000000 0.000000 0.000000 0.000000 0.000000
```

```
0.000000 0.000000 0.000000
```

```
0.200000 0.400000 0.600000 0.800000 0.000000 0.000000 0.000000 0.000000 0.000000
```

```
0.000000 0.000000 0.000000
```

```
0.100000 0.200000 0.300000 0.400000 0.500000 0.600000 0.700000 0.800000 0.000000
```

```
0.000000 0.000000 0.000000
```

(3) vld3 和 vst3

vld3 指令用于从内存加载数据到 3 个向量, 其内联函数定义如下:

```
Result_t vld3_type ( Scalar_t* N );
```

```
Result_t vld3q_type ( Scalar_t* N );
```


vst3 指令实现存储 3 个向量到内存，其内联函数定义如下：

```
void vst3_type ( Scalar_t* N, Vector_t M );
```

```
void vst3q_type ( Scalar_t* N, Vector_t M );
```

通过 vld3 和 vst3 指令可以实现对数据的解交织（De-interleaving）和交织（Interleaving）操作，如图 8-10 所示。

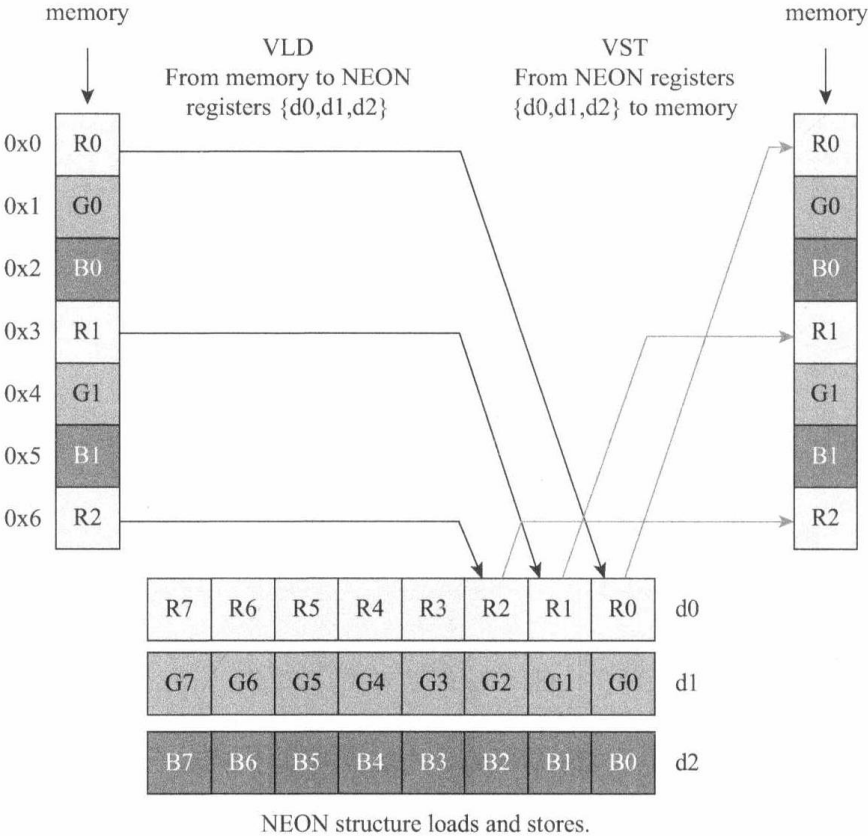


图 8-10 解交织和交织操作^①

以具体的内联函数 vld3q_f32 和 vst3q_f32 为例，其完整测试代码如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <arm_neon.h>
#include <math.h>

int main ( )
{
    float src [ 12 ] = {0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 0.10, 0.11, 0.12};
    float dst [ 12 ] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
```

① 图来源：NEON Programmer's Guide Version: 1.0, Figure 6-4

```

float dst_0 [ 12 ]= {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
float dst_1 [ 12 ]= {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
float dst_2 [ 12 ]= {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
float32x4x3_t vec0 = vld3q_f32 ( src );

vst1q_f32 ( dst_0, vec0.val [ 0 ] ); //float32x4_t
vst1q_f32 ( dst_1, vec0.val [ 1 ] );
vst1q_f32 ( dst_2, vec0.val [ 2 ] );
for ( int i = 0 ; i < 12; i++ ){
    printf ( " %f" , dst_0 [ i ] );
}
printf ( " \n" );
for ( int i = 0 ; i < 12; i++ ){
    printf ( " %f" , dst_1 [ i ] );
}
printf ( " \n" );
for ( int i = 0 ; i < 12; i++ ){
    printf ( " %f" , dst_2 [ i ] );
}
printf ( " \n" );

vst3q_f32 ( dst, vec0 );
for ( int i = 0 ; i < 12; i++ ){
    printf ( " %f" , dst [ i ] );
}
printf ( " \n" );
}

```

代码运行结果:

```

[ root@ATK-DLRV1126:/ ] # ./test_neon
0.100000 0.400000 0.700000 0.100000 0.000000 0.000000 0.000000 0.000000 0.000000
0.000000 0.000000 0.000000
0.200000 0.500000 0.800000 0.110000 0.000000 0.000000 0.000000 0.000000 0.000000
0.000000 0.000000 0.000000
0.300000 0.600000 0.900000 0.120000 0.000000 0.000000 0.000000 0.000000 0.000000

```

0.000000 0.000000 0.000000

0.100000 0.200000 0.300000 0.400000 0.500000 0.600000 0.700000 0.800000 0.900000

0.100000 0.110000 0.120000

(4) vld4 和 vst4

vld4 指令用于从内存加载数据到 4 个向量，其内联函数定义如下：

```
Result_t vld4_type ( Scalar_t* N );
```

```
Result_t vld4q_type ( Scalar_t* N );
```

vst4 指令实现存储 4 个向量到内存，其内联函数定义如下：

```
void vst4_type ( Scalar_t* N, Vector_t M );
```

```
void vst4q_type ( Scalar_t* N, Vector_t M );
```

以具体的内联函数 vld4q_f32 和 vst4q_f32 为例，其完整测试代码如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <arm_neon.h>
#include <math.h>

int main ( )
{
    float src [ 16 ] = {0.1, 0.2, 0.3, 0.4, 0.5, 0.6, 0.7, 0.8, 0.9, 0.10, 0.11,
0.12, 0.13, 0.14, 0.15, 0.16};

    float dst [ 16 ] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
    float dst_0 [ 12 ] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
    float dst_1 [ 12 ] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
    float dst_2 [ 12 ] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
    float dst_3 [ 12 ] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};

    float32x4x4_t vec0 = vld4q_f32 ( src );

    vst1q_f32 ( dst_0, vec0.val [ 0 ] ); // float32x4_t
    vst1q_f32 ( dst_1, vec0.val [ 1 ] );
    vst1q_f32 ( dst_2, vec0.val [ 2 ] );
    vst1q_f32 ( dst_3, vec0.val [ 3 ] );

    for ( int i = 0; i < 12; i++ ) {
        printf ( " %f" , dst_0 [ i ] );
    }
```

```

printf ( " \n" );
for ( int i = 0; i < 12; i++ ) {
    printf ( " %f" , dst_1 [ i ] );
}
printf ( " \n" );
for ( int i = 0; i < 12; i++ ) {
    printf ( " %f" , dst_2 [ i ] );
}
printf ( " \n" );
for ( int i = 0; i < 12; i++ ) {
    printf ( " %f" , dst_3 [ i ] );
}
printf ( " \n" );

vst4q_f32 ( dst, vec0 );
for ( int i = 0; i < 16; i++ ) {
    printf ( " %f" , dst [ i ] );
}
printf ( " \n" );
}

```

代码运行结果:

```

[ root@ATK-DLRV1126:/ ] # ./test_neon
0.100000 0.500000 0.900000 0.130000 0.000000 0.000000 0.000000 0.000000 0.000000
0.000000 0.000000 0.000000
0.200000 0.600000 0.100000 0.140000 0.000000 0.000000 0.000000 0.000000 0.000000
0.000000 0.000000 0.000000
0.300000 0.700000 0.110000 0.150000 0.000000 0.000000 0.000000 0.000000 0.000000
0.000000 0.000000 0.000000
0.400000 0.800000 0.120000 0.160000 0.000000 0.000000 0.000000 0.000000 0.000000
0.000000 0.000000 0.000000
0.100000 0.200000 0.300000 0.400000 0.500000 0.600000 0.700000 0.800000 0.900000
0.100000 0.110000 0.120000

```

8.4.3 算术运算指令

算术运算指令（Arithmetic）实现对整数和浮点数向量进行加减运算。文中我们以加法运算为例介绍。加法的内联函数定义如下：

Result_t vadd_type (Vector1_t N, Vector2_t M);

Result_t vaddq_type (Vector1_t N, Vector2_t M);

其中符号 Result_t、type、Vector1_t、Vector2_t 的数据类型选项如表 8-14 所示：

表 8-14 算术运算指令数据类型选项

D ₀ —D ₃₁ 寄存器 64 bit			
Result_t	type	Vector1_t	Vector2_t
int8x8_t	s8	int8x8_t	int8x8_t
int16x4_t	s16	int16x4_t	int16x4_t
int32x2_t	s32	int32x2_t	int32x2_t
int64x1_t	s64	int64x1_t	int64x1_t
uint8x8_t	u8	uint8x8_t	uint8x8_t
uint16x4_t	u16	uint16x4_t	uint16x4_t
uint32x2_t	u32	uint32x2_t	uint32x2_t
uint64x1_t	u64	uint64x1_t	uint64x1_t
float32x2_t	f32	float32x2_t	float32x2_t
Q ₀ —Q ₁₅ 寄存器 128 bit			
Result_t	type	Vector1_t	Vector2_t
int8x16_t	s8	int8x16_t	int8x16_t
int16x8_t	s16	int16x8_t	int16x8_t
int32x4_t	s32	int32x4_t	int32x4_t
int64x2_t	s64	int64x2_t	int64x2_t
uint8x16_t	u8	uint8x16_t	uint8x16_t
uint16x8_t	u16	uint16x8_t	uint16x8_t
uint32x4_t	u32	uint32x4_t	uint32x4_t
uint64x2_t	u64	uint64x2_t	uint64x2_t
float32x4_t	f32	float32x4_t	float32x4_t

以内联函数 vadd_u8 为例，完整测试代码如下：

示例代码 8.4.3.1 vadd_u8 测试代码

```

#include <stdio.h>
#include <stdlib.h>
#include <arm_neon.h>
#include <math.h>

int main ( )
{
    uint8_t vector_0 [ 8 ] = {1, 2, 3, 4, 5, 6, 7, 8};
    uint8_t vector_1 [ 8 ] = {11, 12, 13, 14, 15, 16, 17, 18};
    uint8_t result [ 8 ];

    uint8x8_t vec0 = vld1_u8 ( vector_0 );
    // 从数组 vector_0 中加载 8 个 uint8 类型元素存到寄存器 vec0 中
    uint8x8_t vec1 = vld1_u8 ( vector_1 );
    uint8x8_t vec_r = vadd_u8 ( vec0, vec1 );
    vst1_u8 ( result, vec_r );
    // 打印结果
    for( int i = 0; i < 8; i++ ){
        printf ( " %d" , result [ i ] );
    }
    printf ( " \n" );
}

```

代码运行结果：

```
[root@ATK-DLRV1126:/] # ./test_neon
```

```
12 14 16 18 20 22 24 26
```

以内联函数 vadd_s8 为例，完整测试代码如下：

```

#include <stdio.h>
#include <stdlib.h>
#include <arm_neon.h>
#include <math.h>

int main ( )
{
    int8_t vector_0 [ 8 ] = {1, 2, 3, 4, 5, 6, 7, 8};

```

```
int8_t vector_1 [ 8 ] = {11, 12, 13, 14, 15, 16, 17, 18};
int8_t result [ 8 ];

int8x8_t vec0 = vld1_s8 ( vector_0 );
// 从数组 vector_0 中 load 8 个元素存到寄存器 vec0 中
int8x8_t vec1 = vld1_s8 ( vector_1 );
int8x8_t vec_r = vadd_s8 ( vec0, vec1 );
vst1_s8 ( result, vec_r );
// 打印结果
for( int i = 0; i < 8; i++ ){
    printf ( " %d" , result [ i ] );
}
printf ( " \n" );
}
```

代码运行结果：

```
[ root@ATK-DLRV1126:/ ] # ./test_neon
12 14 16 18 20 22 24 26
```

8.4.4 数据处理指令

数据处理指令（Data processing）对通用的数据进行处理操作。

（1）vabs

vabs 指令实现对向量中每个 8、16 或 32 位元素的绝对值求解，其内联函数定义如下：

```
Result_t vabs_type ( Vector_t N );
Result_t vabsq_type ( Vector_t N );
具体的参数选项如表 8-15 所示。
```

表 8-15 vabs 指令具体参数选项

vabs 参数选项		
Result_t	type	Vector_t
int8x8_t	s8	int8x8_t
int16x4_t	s16	int16x4_t
int32x2_t	s32	int32x2_t
float32x2_t	f32	float32x2_t

续表

vabs 参数选项		
Result_t	type	Vector_t
int8x16_t	s8	int8x16_t
int16x8_t	s16	int16x8_t
int32x4_t	s32	int32x4_t
float32x4_t	f32	float32x4_t

(2) vneg

vneg 指令对向量中每个 8、16 或 32 位元素取反，其内联函数定义如下：

Result_t vneg_type (Vector_t N);

Result_t vnegq_type (Vector_t N);

具体的参数选项如表 8-16 所示。

表 8-16 vneg 指令参数选项

vneg 参数选项		
Result_t	type	Vector_t
int8x8_t	s8	int8x8_t
int16x4_t	s16	int16x4_t
int32x2_t	s32	int32x2_t
float32x2_t	f32	float32x2_t
vnegq 参数选项		
Result_t	type	Vector_t
int8x16_t	s8	int8x16_t
int16x8_t	s16	int16x8_t
int32x4_t	s32	int32x4_t
float32x4_t	f32	float32x4_t

(3) vmax 和 vmin

vmax 和 vmin 指令对两个向量中对应位置的 8、16 或 32 位元素依次比较大小，返回最大或最小值，其内联函数定义如下：

Result_t vmax_type (Vector1_t N, Vector2_t M);

Result_t vmaxq_type (Vector1_t N, Vector2_t M);

具体的参数选项如表 8-17 所示。

表 8-17 vmax 指令参数选项

vmax 参数选项			
Result_t	type	Vector1_t	Vector2_t
int8x8_t	s8	int8x8_t	int8x8_t
int16x4_t	s16	int16x4_t	int16x4_t
int32x2_t	s32	int32x2_t	int32x2_t
uint8x8_t	u8	uint8x8_t	uint8x8_t
uint16x4_t	u16	uint16x4_t	uint16x4_t
uint32x2_t	u32	uint32x2_t	uint32x2_t
float32x2_t	f32	float32x2_t	float32x2_t
vmaxq 参数选项			
Result_t	type	Vector1_t	Vector2_t
int8x16_t	s8	int8x16_t	int8x16_t
int16x8_t	s16	int16x8_t	int16x8_t
int32x4_t	s32	int32x4_t	int32x4_t
uint8x16_t	u8	uint8x16_t	uint8x16_t
uint16x8_t	u16	uint16x8_t	uint16x8_t
uint32x4_t	u32	uint32x4_t	uint32x4_t
float32x4_t	f32	float32x4_t	float32x4_t

(4) vrecpe

vrecpe 指令用于求向量中每个元素的近似倒数，其内联函数定义如下：

```
Result_t vrecpe_type ( Vector_t N );
```

```
Result_t vrecpeq_type ( Vector_t N );
```

其中 type 可选为 u32 或 f32。

(5) vrecps

vrecps 指令用于计算向量中每个元素的倒数，其内联函数定义如下：

```
float32x2_t vrecps_f32 ( float32x2_t N, float32x2_t M );
```

```
float32x4_t vrecpsq_f32 ( float32x4_t N, float32x4_t M );
```

(6) vrsqrte

vrsqrte 指令用于求向量中每个元素的平方根，其内联函数定义如下：

```
Result_t vrsqrte_type ( Vector_t N );
```

```
Result_t vrsqrteq_type ( Vector_t N );
```

其中 type 可选为 u32 或 f32。

(7) vcnt

vcnt 指令用于统计向量中每个元素中 bit 为 1 的个数，其内联函数定义如下：

```
Result_t vcnt_type ( Vector_t N );
```

```
Result_t vcntq_type ( Vector_t N );
```

8.4.5 向量乘法指令

向量乘法指令 (Multiply) 用于实现整型或浮点型向量的乘法运算。

(1) vmul

vmul 指令用于实现两个向量中的对应元素相乘，也叫点乘，其内联函数定义如下：

```
Result_t vmul_type ( Vector1_t N, Vector2_t M );
```

```
Result_t vmulq_type ( Vector1_t N, Vector2_t M );
```

(2) vmla

vmla (Vector Multiply-Add) 指令实现将第二个和第三个输入向量中的对应元素点乘，然后将乘积加到第一个输入向量中的对应元素上，其内联函数定义如下：

```
Result_t vmla_type ( Vector1_t N, Vector2_t M, Vector3_t P );
```

```
Result_t vmlaq_type ( Vector1_t N, Vector2_t M, Vector3_t P );
```

8.4.6 逻辑和比较运算指令

逻辑运算指令 (Logical) 用于实现与、或、非等运算。

比较运算指令 (Compare) 用于实现等于、大于、小于等运算。

8.4.7 浮点指令

浮点指令 (Floating-point) 用于实现单精度浮点数、半精度浮点数、32 位整型数 (Fixed-point、Integer) 之间的转换操作。

(1) vcvtn_f32

vcvt_n_f32 指令用于实现定点数转浮点数，其内联函数定义^①如下：

```
Result_t vcvtn_f32_type ( Vector_t N, int n );
```

```
Result_t vcvtnq_f32_type ( Vector_t N, int n );
```

其中符号 Result_t、type、Vector_t 的数据类型选项如表 8-18 所示：

^① Vector Convert

表 8-18 vcvtn_f32 指令数据类型选项

D ₀ —D ₃₁ 寄存器 64 bit			
Result_t	type	Vector_t	int range
float32x2_t	s32	int32x2_t	1—32
float32x2_t	u32	uint32x2_t	1—32
Q ₀ —Q ₁₅ 寄存器 128 bit			
Result_t	type	Vector_t	int range
float32x4_t	s32	int32x4_t	1—32
float32x4_t	u32	uint32x4_t	1—32

以具体的内联函数 `vcvtn_f32_s32` 为例，其完整的测试代码如下：

Code 8-1: `vcvtn_f32_s32-neon.cpp`

```
#include <arm_neon.h>
```

```
#include <stdio.h>
```

```
int main ( )
```

```
{
```

```
    int src [ 12 ] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 };
```

```
    int32x2_t vec0 = vld1_s32 ( src );
```

```
    // vld1_s32 一次加载 src 地址起始的 2 个 int32 浮点数放到向量寄存器 vec0 中
```

```
    for ( int i = 0 ; i < 12; i++ )
```

```
    {
```

```
        printf ( " %d" , src [ i ] );
```

```
    }
```

```
    printf ( " \n\n" );
```

```
    //*****
```

```
    float32x2_t vec1 = vcvtn_f32_s32 ( vec0, 1 );
```

```
    float dst1 [ 12 ] = { 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0 };
```

```
    vst1_f32 ( dst1, vec1 );
```

```
    for ( int i = 0 ; i < 12; i++ )
```

```
    {
```

```
        printf ( " %f" , dst1 [ i ] );
```

```
    }
```

```
    printf ( " \n" );
```

```

//*****
float32x2_t vec2 = vcvtn_f32_s32 ( vec0, 2 );
float dst2 [ 12 ] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
vst1_f32 ( dst2, vec2 );
for ( int i = 0 ; i < 12; i++ )
{
    printf ( " %f" , dst2 [ i ] );
}
printf ( " \n" );
//*****
float32x2_t vec3 = vcvtn_f32_s32 ( vec0, 3 );
float dst3 [ 12 ] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
vst1_f32 ( dst3, vec3 );
for ( int i = 0 ; i < 12; i++ )
{
    printf ( " %f" , dst3 [ i ] );
}
printf ( " \n" );
//*****
float32x2_t vec4 = vcvtn_f32_s32 ( vec0, 4 );
float dst4 [ 12 ] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
vst1_f32 ( dst4, vec4 );
for ( int i = 0 ; i < 12; i++ )
{
    printf ( " %f" , dst4 [ i ] );
}
printf ( " \n" );

return 0;
}

```

代码运行结果:

```
[ root@ATK-DLRV1126:/ ] # ./test_neon
```

```
1 2 3 4 5 6 7 8 9 10 11 12
```

```
0.500000 1.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
```

```
0.000000 0.000000 0.000000
    0.250000 0.500000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
0.000000 0.000000 0.000000
    0.125000 0.250000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
0.000000 0.000000 0.000000
    0.062500 0.125000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
0.000000 0.000000 0.000000
```

(2) vcvtn

vcvtn 实现浮点数转定点数，其内联函数定义如下：

```
Result_t vcvtn_type_f32 ( Vector_t N, int n );
```

```
Result_t vcvtnq_n_type_f32 ( Vector_t N, int n );
```

其中符号 Result_t、type、Vector_t 的数据类型选项如表 8-19 所示：

表 8-19 vcvtn 数据类型选项

D ₀ —D ₃₁ 寄存器 64 bit			
Result_t	type	Vector_t	int range
int32x2_t	s32	float32x2_t	1—32
uint32x2_t	u32	float32x2_t	1—32
Q ₀ —Q ₁₅ 寄存器 128 bit			
Result_t	type	Vector_t	int range
int32x4_t	s32	float32x4_t	1—32
uint32x4_t	u32	float32x4_t	1—32

以内联函数 vcvtn_s32_f32 为例，其完整测试代码如下：

```
#include <arm_neon.h>
#include <stdio.h>

int main ( )
{
    float src [ 12 ] = { 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12 };
    float32x2_t vec0 = vld1_f32 ( src );
    // vld1_f32 一次加载 src 地址起始的 2 个 float32 浮点数放到向量寄存器 vec0 中
    for ( int i = 0; i < 12; i++ )
    {
```



```

    printf ( " %f" , src [ i ] );
}
printf ( " \n\n" );
/*****
int32x2_t vec1 = vcvtn_s32_f32 ( vec0, 1 );
int dst1 [ 12 ] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
vst1_s32 ( dst1, vec1 );
for ( int i = 0; i < 12; i++ )
{
    printf ( " %d" , dst1 [ i ] );
}
printf ( " \n" );
/*****
int32x2_t vec2 = vcvtn_s32_f32 ( vec0, 2 );
int dst2 [ 12 ] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
vst1_s32 ( dst2, vec2 );
for ( int i = 0; i < 12; i++ )
{
    printf ( " %d" , dst2 [ i ] );
}
printf ( " \n" );
/*****
int32x2_t vec3 = vcvtn_s32_f32 ( vec0, 3 );
int dst3 [ 12 ] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
vst1_s32 ( dst3, vec3 );
for ( int i = 0; i < 12; i++ )
{
    printf ( " %d" , dst3 [ i ] );
}
printf ( " \n" );
/*****
int32x2_t vec4 = vcvtn_s32_f32 ( vec0, 4 );
int dst4 [ 12 ] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
vst1_s32 ( dst4, vec4 );

```

```
for ( int i = 0; i < 12; i++ )
{
    printf ( " %d" , dst4 [ i ] );
}
printf ( " \n" );

return 0;
}
```

代码运行结果：

```
[root@ATK-DLRV1126:/] # ./test_neon
1.000000 2.000000 3.000000 4.000000 5.000000 6.000000 7.000000 8.000000 9.000000
10.000000 11.000000 12.000000
```

```
2400000000000
4800000000000
81600000000000
163200000000000
```

(3) vcvt_f32

vcvt_f32 实现定点数转浮点数，其内联函数定义如下：

```
Result_t vcvt_f32_type ( Vector_t N );
```

```
Result_t vcvtq_f32_type ( Vector_t N );
```

其中符号 Result_t、type、Vector_t 的数据类型选项如表 8-20 所示：

表 8-20 vcvt_f32 数据类型选项

D ₀ —D ₃₁ 寄存器 64 bit		
Result_t	type	Vector_t
float32x2_t	s32	int32x2_t
float32x2_t	u32	uint32x2_t
Q ₀ —Q ₁₅ 寄存器 128 bit		
Result_t	type	Vector_t
float32x4_t	s32	int32x4_t
float32x4_t	u32	uint32x4_t

以具体的内联函数 `vcvt_f32_s32` 为例，其完整的测试代码如下：

```
#include <arm_neon.h>
#include <stdio.h>

int main ( )
{
    int src [ 12 ] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12};
    int32x2_t vec0 = vld1_s32 ( src );
    // vld1_s32 一次加载 src 地址起始的 2 个 int32 整型数据放到向量寄存器 vec0 中
    for ( int i = 0 ; i < 12; i++ )
    {
        printf ( " %d" , src [ i ] );
    }
    printf ( " \n\n" );

    float32x2_t vec1 = vcvt_f32_s32 ( vec0 );
    float dst [ 12 ] = {0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0};
    vst1_f32 ( dst, vec1 );
    for ( int i = 0; i < 12; i++ )
    {
        printf ( " %f" , dst [ i ] );
    }
    printf ( " \n\n" );
    return 0;
}
```

代码分析：

函数 `float32x2_t vec1 = vcvt_f32_s32 ( vec0 )` 实现 2 个 int32 整型数转为 float32 浮点数。

代码运行结果：

```
[root@ATK-DLRV1126:/] # ./test_neon
```

```
1 2 3 4 5 6 7 8 9 10 11 12
```

```
1.000000 2.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000 0.000000
0.000000 0.000000 0.000000
```

8.4.8 移位指令

移位指令（Shift）实现向量位移操作。

8.4.9 置换指令

置换指令（Permutation）实现对向量的顺序重排列操作，不修改数据的值。

（1）vrev

vrev（Reverse）为反转元素指令，包括 vrev16、vrev32、vrev64。

vrev16 指令将向量中的每个半字（16 bit）内的 8 位元素顺序反转，并将结果存储到对应的目标向量中，如内联函数 `int8x8_t vrev16_s8 (int8x8_t N)`，实现示意如图 8-11 所示。

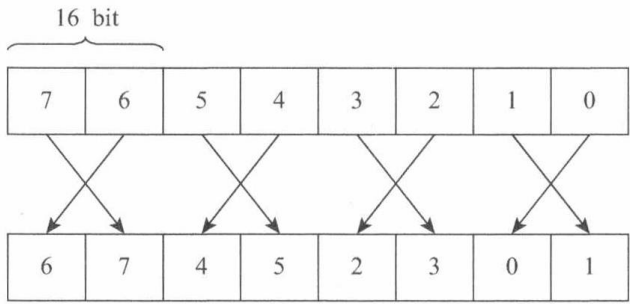


图 8-11 vrev16 指令反转示意图

vrev32 指令将向量中的每个字（32 bit）内的 8 位或 16 位元素的顺序反转，并将结果存储到对应的目标向量中。图 8-12 中，图（a）为内联函数 `int8x8_t vrev32_s8 (int8x8_t N)` 的实现示意，图（b）为内联函数 `int16x4_t vrev32_s16 (int16x4_t N)` 的实现示意。

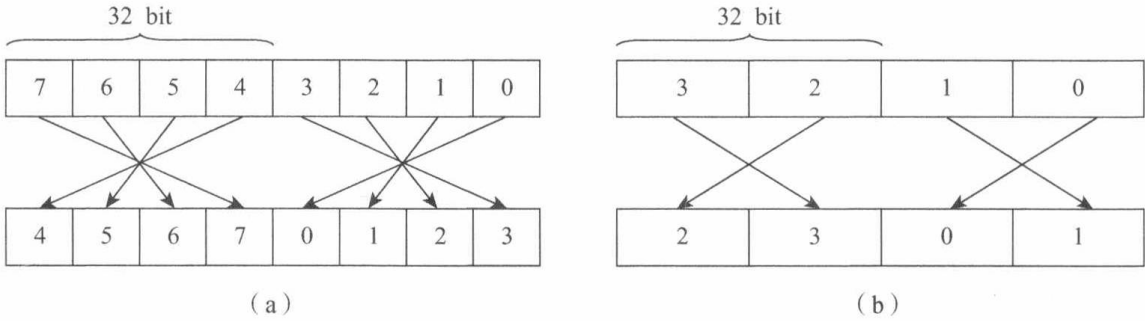


图 8-12 vrev32 指令反转示意图

vrev64 指令将向量中的每个双字（64 bit）内的 8 位、16 位或 32 位的元素顺序反转，并将结果存储到对应的目标向量中，实现示意如图 8-13 所示。

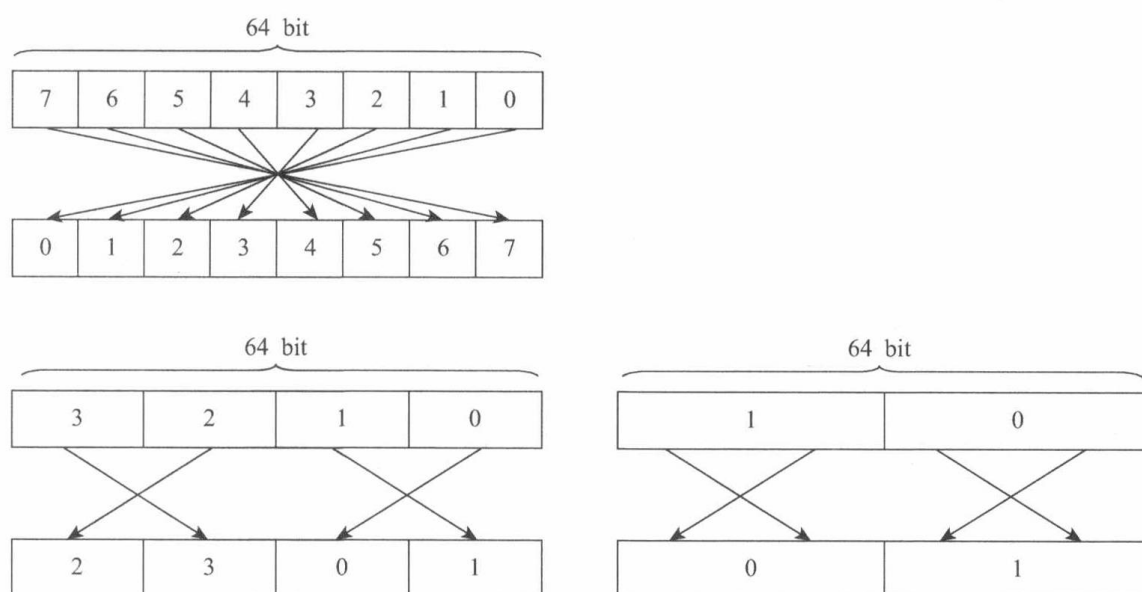
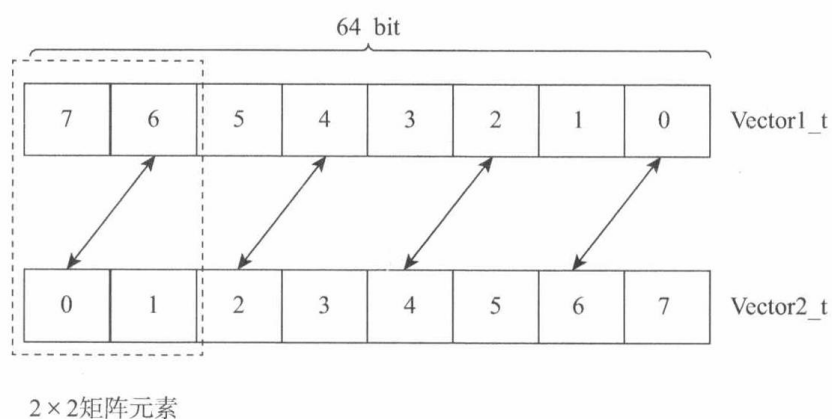


图 8-13 vrev64 指令反转示意图

(2) vtrn

vtrn (Transpose) 为转置指令, 可实现 2×2 小矩阵的转置操作, 如图 8-14 所示。

图 8-14 内联函数 `int8x8x2_t vtrn_s8 (int8x8_t N, int8x8_t M)` 实现 2×2 矩阵元素转置

vtrn 内联函数定义为 `Result_t vtrn_type (Vector1_t N, Vector2_t M)`, 实现将 Vector1 中奇数索引 (索引从 0 开始) 的元素与 Vector2 中偶数索引的元素进行交换。具体的内联函数 `int8x8x2_t vtrn_s8 (int8x8_t N, int8x8_t M)` 测试代码如下:

```
#include <stdio.h>
#include <stdlib.h>
#include <arm_neon.h>

int main ( )
{
    int8_t vector_0 [8] = {1, 2, 3, 4, 5, 6, 7, 8};
```

```

int8_t vector_1 [ 8 ] = { 9, 10, 11, 12, 13, 14, 15, 16 };
int8_t result [ 8 ];

int8x8_t vec0 = vld1_s8 ( vector_0 );
// 从数组 vector_0 中 load 8 个元素存到寄存器 vec0 中
int8x8_t vec1 = vld1_s8 ( vector_1 );
int8x8x2_t vec_r = vtrn_s8 ( vec0, vec1 );
vst1_s8 ( result, vec_r.val [ 0 ] );
for ( int i = 0; i < 8; i++ ) {
    printf ( " %d" , result [ i ] );
}
printf ( " \n" );

vst1_s8 ( result, vec_r.val [ 1 ] );
for ( int i = 0; i < 8; i++ ) {
    printf ( " %d" , result [ i ] );
}
printf ( " \n" );
}

```

代码运行结果：

```

1 9 3 11 5 13 7 15
2 10 4 12 6 14 8 16

```

(3) vtbl

vtbl (Vector Table Lookup) 为查表指令。其内联函数定义如下：

```
Result_t vtbl1_type ( Vector1_t N, Vector2_t M );
```

其中 M 为索引，根据索引在表 N 中查找对应位置的元素，最终输出到新的向量中。如果索引超出范围，则返回 0。表 N 使用一个 D 寄存器。该内联函数的参数选项如表 8-21 所示。

表 8-21 vtbl1 内联函数参数选项

Result_t	type	Vector1_t	Vector2_t
int8x8_t	s8	int8x8_t	int8x8_t
uint8x8_t	u8	uint8x8_t	uint8x8_t
poly8x8_t	p8	poly8x8_t	poly8x8_t

以具体的内联函数 `vtbl1_s8` 为例，其完整的测试代码如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <arm_neon.h>

int main ( )
{
    int8_t vector_0 [ 8 ] = {1, 2, 3, 4, 5, 6, 7, 8};
    int8_t vector_1 [ 8 ] = {7, 6, 5, 4, 3, 2, 0, -1};
    int8_t result [ 8 ];

    int8x8_t vec0 = vld1_s8 ( vector_0 );
    // 从数组 vector_0 中 load 8 个元素存到寄存器 vec0 中
    int8x8_t vec1 = vld1_s8 ( vector_1 );

    int8x8_t vec_r = vtbl1_s8 ( vec0, vec1 );

    vst1_s8 ( result, vec_r );
    for( int i = 0; i < 8; i++ ){
        printf ( " %d" , result [ i ] );
    }
    printf ( " \n" );
}
```

代码运行结果：

8 7 6 5 4 3 1 0

内联函数 `Result_t vtbl2_type ( Vector1_t N, Vector2_t M )`，其中表 N 使用 2 个 D 寄存器，具体的参数选项如表 8-22 所示。

表 8-22 vtbl2 内联函数参数选项

Result_t	type	Vector1_t	Vector2_t
int8x8_t	s8	int8x8x2_t	int8x8_t
uint8x8_t	u8	uint8x8x2_t	uint8x8_t
poly8x8_t	p8	poly8x8x2_t	poly8x8_t

以具体的内联函数 `vtbl2_s8` 为例，其完整的测试代码如下：

```
#include <stdio.h>
```

```

#include <stdlib.h>
#include <arm_neon.h>

int main ( )
{
    int8_t vector_0 [ 8 ] = {11, 12, 13, 14, 15, 16, 17, 18};
    int8_t vector_1 [ 8 ] = {19, 20, 21, 22, 23, 24, 25, 26};
    int8_t vector_2 [ 8 ] = {7, 6, 5, 4, 12, 15, 0, -1};
    int8_t result [ 8 ];

    int8x8x2_t vec0;
    vec0.val [ 0 ] = vld1_s8 ( vector_0 );
    vec0.val [ 1 ] = vld1_s8 ( vector_1 );
    int8x8_t vec1 = vld1_s8 ( vector_2 );

    int8x8_t vec_r = vtbl2_s8 ( vec0, vec1 );

    vst1_s8 ( result, vec_r );
    for ( int i = 0; i < 8; i++ ) {
        printf ( " %d" , result [ i ] );
    }
    printf ( " \n" );
}

```

代码运行结果：

18 17 16 15 23 26 11 0

内联函数 `Result_t vtbl3_type ( Vector1_t N, Vector2_t M )`，其中表 N 使用 3 个 D 寄存器，具体的参数选项如表 8-23 所示。

表 8-23 vtbl3 指令参数选项

Result_t	type	Vector1_t	Vector2_t
int8x8_t	s8	int8x8x3_t	int8x8_t
uint8x8_t	u8	uint8x8x3_t	uint8x8_t
poly8x8_t	p8	poly8x8x3_t	poly8x8_t

以具体的内联函数 `vtbl3_s8` 为例，其完整的测试代码如下：

```
#include <stdio.h>
```

```

#include <stdlib.h>
#include <arm_neon.h>

int main ( )
{
    int8_t vector_0 [ 8 ]= {11, 0, 13, 14, 15, 2, 17, -1};
    int8_t vector_1 [ 8 ]= {19, 20, 21, 0, 23, 24, 25, 26};
    int8_t vector_2 [ 8 ]= {27, 1, 2, 22, 6, 24, -1, 26};
    int8_t vector_index [ 8 ]= {7, 6, 5, 4, 12, 15, 0, -1};
    int8_t result [ 8 ];

    int8x8x3_t vec_table;
    vec_table.val [ 0 ]= vld1_s8 ( vector_0 );
    vec_table.val [ 1 ]= vld1_s8 ( vector_1 );
    vec_table.val [ 2 ]= vld1_s8 ( vector_2 );

    int8x8_t vec_index = vld1_s8 ( vector_index );

    int8x8_t vec_r = vtbl3_s8 ( vec_table, vec_index );

    vst1_s8 ( result, vec_r );
    for ( int i = 0; i < 8; i++ ) {
        printf ( " %d" , result [ i ] );
    }
    printf ( " \n" );
}

```

代码运行结果:

-1 17 2 15 23 26 11 0

内联函数 `Result_t vtbl4_type ( Vector1_t N, Vector2_t M )`，其中表 N 使用 4 个 D 寄存器，具体的参数选项如表 8-24 所示。

表 8-24 vtbl4 指令参数选项

Result_t	type	Vector1_t	Vector2_t
int8x8_t	s8	int8x8x4_t	int8x8_t
uint8x8_t	u8	uint8x8x4_t	uint8x8_t
poly8x8_t	p8	poly8x8x4_t	poly8x8_t

以具体的内联函数 `vtbl4_s8` 为例，其完整的测试代码如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <arm_neon.h>

int main ( )
{
    int8_t vector_0 [ 8 ] = {11, 0, 13, 14, 15, 2, 17, -1};
    int8_t vector_1 [ 8 ] = {19, 20, 21, 0, 23, 24, 25, 26};
    int8_t vector_2 [ 8 ] = {27, 1, 2, 22, 6, 24, -1, 26};
    int8_t vector_3 [ 8 ] = {-3, 1, -2, 22, 6, 5, -1, -9};
    int8_t vector_index [ 8 ] = {7, 25, 26, 4, 12, 15, 0, -1};
    int8_t result [ 8 ];

    int8x8x4_t vec_table;
    vec_table.val [ 0 ] = vld1_s8 ( vector_0 );
    vec_table.val [ 1 ] = vld1_s8 ( vector_1 );
    vec_table.val [ 2 ] = vld1_s8 ( vector_2 );
    vec_table.val [ 3 ] = vld1_s8 ( vector_3 );

    int8x8_t vec_index = vld1_s8 ( vector_index );

    int8x8_t vec_r = vtbl4_s8 ( vec_table, vec_index );

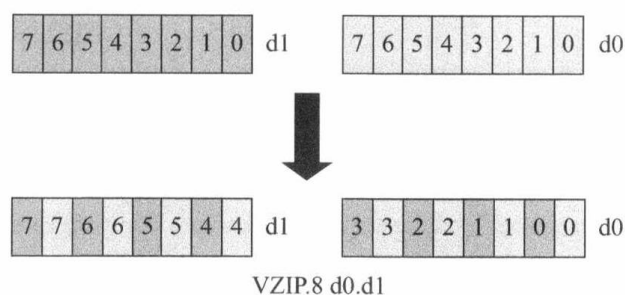
    vst1_s8 ( result, vec_r );
    for ( int i = 0; i < 8; i++ ) {
        printf ( " %d" , result [ i ] );
    }
    printf ( " \n" );
}
```

代码运行结果：

-1 1 2 15 23 26 11 0

(4) `vzip` 和 `vuzp`

`vzip` (Vector Zip) 为压缩指令，实现两个向量的 8、16 或 32 位元素交错组合 (Interleave)，具体就是先按列读后按行写。`vzip` 不同于 `vst2`，前者的结果保存在寄存器中，后者的结果保存在内存中，如图 8-15 所示。

图 8-15 vzip 操作^①

vzip 内联函数定义如下：

Result_t vzip_type (Vector1_t N, Vector2_t M), 具体的参数选项如表 8-25 所示。

表 8-25 vzip 指令参数选项

Result_t	type	Vector1_t	Vector2_t
int8x8x2_t	s8	int8x8_t	int8x8_t
int16x4x2_t	s16	int16x4_t	int16x4_t
int32x2x2_t	s32	int32x2_t	int32x2_t
uint8x8x2_t	u8	uint8x8_t	uint8x8_t
uint16x4x2_t	u16	uint16x4_t	uint16x4_t
uint32x2x2_t	u32	uint32x2_t	uint32x2_t
poly8x8x2_t	p8	poly8x8_t	poly8x8_t
poly16x4x2_t	p16	poly16x4_t	poly16x4_t
float32x2x2_t	f32	float32x2_t	float32x2_t

vzipq 的内联函数定义如下：

Result_t vzipq_type (Vector1_t N, Vector2_t M), 具体的参数选项如表 8-26 所示。

表 8-26 vzipq 指令参数选项

Result_t	type	Vector1_t	Vector2_t
int8x16x2_t	s8	int8x16_t	int8x16_t
int16x8x2_t	s16	int16x8_t	int16x8_t
int32x4x2_t	s32	int32x4_t	int32x4_t
uint8x16x2_t	u8	uint8x16_t	uint8x16_t
uint16x8x2_t	u16	uint16x8_t	uint16x8_t

① 图来源：<https://community.arm.com/arm-community-blogs/b/architectures-and-processors-blog/posts/coding-for-neon---part-5-rearranging-vectors>

续表

Result_t	type	Vector1_t	Vector2_t
uint32x4x2_t	u32	uint32x4_t	uint32x4_t
poly8x16x2_t	p8	poly8x16_t	poly8x16_t
poly16x8x2_t	p16	poly16x8_t	poly16x8_t
float32x4x2_t	f32	float32x4_t	float32x4_t

vuzp (Vector Unzip) 为解压指令，实现两个向量的 8、16 或 32 位元素解交错组合 (de-interleave)，具体就是先按行读后按列写。vuzp 不同于 vld2，前者实现从寄存器到寄存器的数据操作，后者实现从内存加载数据到寄存器，如图 8-16 所示。

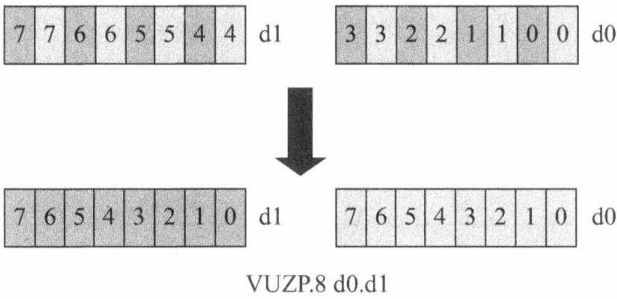


图 8-16 vuzp 操作^①

vuzp 的内联函数定义如下：

Result_t vuzp_type (Vector1_t N, Vector2_t M)，具体的参数选项如表 8-27 所示。

表 8-27 vuzp 指令参数选项

Result_t	type	Vector1_t	Vector2_t
int8x8x2_t	s8	int8x8_t	int8x8_t
int16x4x2_t	s16	int16x4_t	int16x4_t
int32x2x2_t	s32	int32x2_t	int32x2_t
uint8x8x2_t	u8	uint8x8_t	uint8x8_t
uint16x4x2_t	u16	uint16x4_t	uint16x4_t
uint32x2x2_t	u32	uint32x2_t	uint32x2_t
poly8x8x2_t	p8	poly8x8_t	poly8x8_t
poly16x4x2_t	p16	poly16x4_t	poly16x4_t
float32x2x2_t	f32	float32x2_t	float32x2_t

^① 图来源：<https://community.arm.com/arm-community-blogs/b/architectures-and-processors-blog/posts/coding-for-neon---part-5-rearranging-vectors>

vuzpq 的内联函数定义如下：

Result_t vuzpq_type (Vector1_t N, Vector2_t M), 具体的参数选项如表 8-28 所示。

表 8-28 vuzpq 指令参数选项

Result_t	type	Vector1_t	Vector2_t
int8x16x2_t	s8	int8x16_t	int8x16_t
int16x8x2_t	s16	int16x8_t	int16x8_t
int32x4x2_t	s32	int32x4_t	int32x4_t
uint8x16x2_t	u8	uint8x16_t	uint8x16_t
uint16x8x2_t	u16	uint16x8_t	uint16x8_t
uint32x4x2_t	u32	uint32x4_t	uint32x4_t
poly8x16x2_t	p8	poly8x16_t	poly8x16_t
poly16x8x2_t	p16	poly16x8_t	poly16x8_t
float32x4x2_t	f32	float32x4_t	float32x4_t

8.4.10 其他指令

其他指令 (Miscellaneous) 包括一些杂项的指令，比如 vcreate、vdup_n、vdup_lane、vmove_n 等。

(1) vcreate

vcreate 指令实现创建一个 64bit 的向量，内联函数定义如下：

Result_t vcreate_type (Scalar_t N);

其中符号 Result_t、type、Scalar_t 的数据类型选项如表 8-29 所示：

表 8-29 vcreate 数据类型选项

Result_t	type	Scalar_t
int8x8_t	s8	uint64_t
int16x4_t	s16	uint64_t
int32x2_t	s32	uint64_t
int64x1_t	s64	uint64_t
uint8x8_t	u8	uint64_t
uint16x4_t	u16	uint64_t
uint32x2_t	u32	uint64_t
uint64x1_t	u64	uint64_t
float16x4_t	fl16	uint64_t

续表

Result_t	type	Scalar_t
float32x2_t	f32	uint64_t
poly8x8_t	p8	uint64_t
poly16x4_t	p16	uint64_t

以内联函数 `vcreate_u16` 为例，其完整测试代码如下：

```
#include <stdio.h>
#include <stdlib.h>
#include <arm_neon.h>

int main( void )
{
    uint16x4_t vector; // define v as a vector with 4 lanes of 16-bit data
    unsigned short result [ 4 ]; // 分配 4 个 16bit 的内存块
    vector = vcreate_u16 ( 0x000A000301020012 );
    vst1_u16 ( result, vector ); // store the vector to memory, in this case, to array result

    for ( int i=0; i<4; i++ )
        printf ( " %d" , result [ i ] );
    printf ( " \n" );
    unsigned char c = result [ 1 ];
    printf ( " %d" , c );
    printf ( " \n" );
}
```

代码分析：

函数 `vector = vcreate_u16 ( 0x000A000301020012 )` 创建一个向量，其中的值从低位到高位分别为 18 258 3 10，如图 8-17 所示。

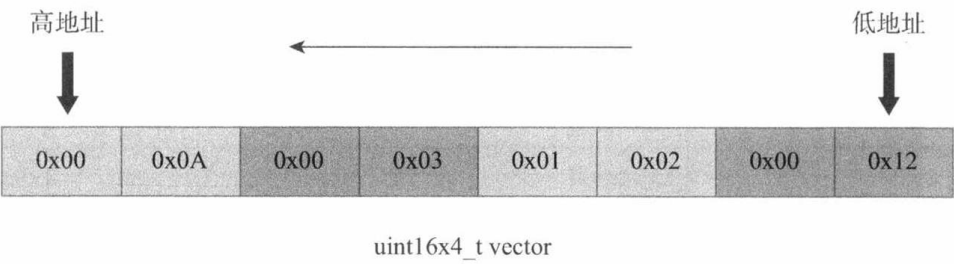


图 8-17 `vcreate_u16`

编译方法：

```
$ arm-linux-gnueabi-g++ neon-vcreate_u8.cpp -g -o test_neon
```

代码运行结果：

```
[root@ATK-DLRV1126:/] # ./test_neon
```

```
18 258 3 10
```

```
2
```

(2) vdup_n

vdup_n 指令将标量复制到目标向量的每个元素中，其内联函数定义如下：

```
Result_t vdup_n_type ( Scalar_t N );
```

```
Result_t vdupq_n_type ( Scalar_t N );
```

(3) vdup_lane

vdup_lane 指令将标量复制到目标向量的每个元素中，其内联函数定义如下：

```
Result_t vdup_lane_type ( Vector_t N, int n );
```

```
Result_t vdupq_lane_type ( Vector_t N, int n );
```

(4) vmov_n

vmov_n 指令将标量复制到目标向量的每个元素中，其内联函数定义如下：

```
Result_t vmov_n_type ( Scalar_t N );
```

```
Result_t vmovq_n_type ( Scalar_t N );
```

8.5 Neon 编程优化算法实例

8.5.1 RGB 转 Gray 颜色空间

RGB 转 Gray（灰度图）是很多图像算法的组成部分，其中 RGB 转 Gray 的心理学计算公式为：

$$\text{gray} = r \times 0.299 + g \times 0.587 + b \times 0.114 \quad (8.5-1)$$

式 (8.5-1) 中由于采用浮点计算，往往性能没有定点高。于是可以优化为采用 16 位定点数表示和移位计算实现除法，具体见式 (8.5-2)。

$$\text{gray} = \left\{ \begin{array}{l} r \times \text{int}(0.299 \times 2^{16}) + g \times \text{int}(0.587 \times 2^{16}) + \\ b \times [2^{16} - \text{int}(0.299 \times 2^{16}) - \text{int}(0.587 \times 2^{16})] \end{array} \right\} \gg 16 \quad (8.5-2)$$

$$\text{gray} = (r \times 19595 + g \times 38469 + b \times 7472) \gg 16$$

还可以进一步压缩到 8 位定点数表示，见式 (8.5-3)。

$$\text{gray} = \left\{ \begin{array}{l} r \times \text{int}(0.299 \times 2^8) + g \times \text{int}(0.587 \times 2^8) + \\ b \times [2^8 - \text{int}(0.299 \times 2^8) - \text{int}(0.587 \times 2^8)] \end{array} \right\} \gg 8 \quad (8.5-3)$$

$\text{gray} = [r \times 76 + g \times 150 + b \times (256 - 76 - 150)] \gg 8$

$\text{gray} = (r \times 76 + g \times 150 + b \times 30) \gg 8$

$2^8 = 256$

以上公式用 C 语言代码实现如下：

```
for ( i=0; i<n; i++) {
    int r = *src++;
    int g = *src++;
    int b = *src++;
    int y = ( r*19595 ) + ( g*38469 ) + ( b*7472 );
    *dest++ = ( y>>16 );
}
```

Neon 编程代码如下：

```
// 读取 8 字节预设值到 64 位寄存器
uint8x8_t rfac = vdup_n_u8 ( 76 );
uint8x8_t gfac = vdup_n_u8 ( 150 );
uint8x8_t bfac = vdup_n_u8 ( 30 );
n/=8;
for ( i=0; i<n; i++)
{
    uint16x8_t temp;
    // 一次读取 3 个 unit8x8 到 3 个 64 位寄存器
    uint8x8x3_t rgb = vld3_u8 ( src );
    uint8x8_t result;

    temp = vmull_u8 ( rgb.val [ 0 ], rfac );
    temp = vmlal_u8 ( temp, rgb.val [ 1 ], gfac );
    temp = vmlal_u8 ( temp, rgb.val [ 2 ], bfac );
    result = vshrn_n_u16 ( temp, 8 );
    vst1_u8 ( dest, result );
    src += 8*3;
    dest += 8;
}
```

其中内联函数 `vld3q_u8(...)` 的实现如图 8-18 所示：

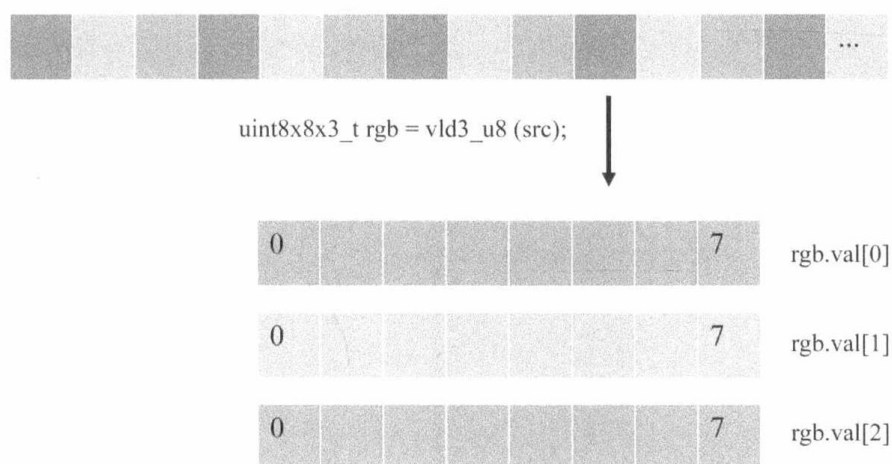


图 8-18 vld3q_u8(...) 操作示意图

8.5.2 RGB 内存空间解交织和交织

`vld3q_u8(...)` 实现 RGB 内存数据排列顺序解交织 (HWC→CHW 解交织)，也叫打包 (Packed) 格式和平面 (Planar) 格式的转换，如图 8-19 所示。

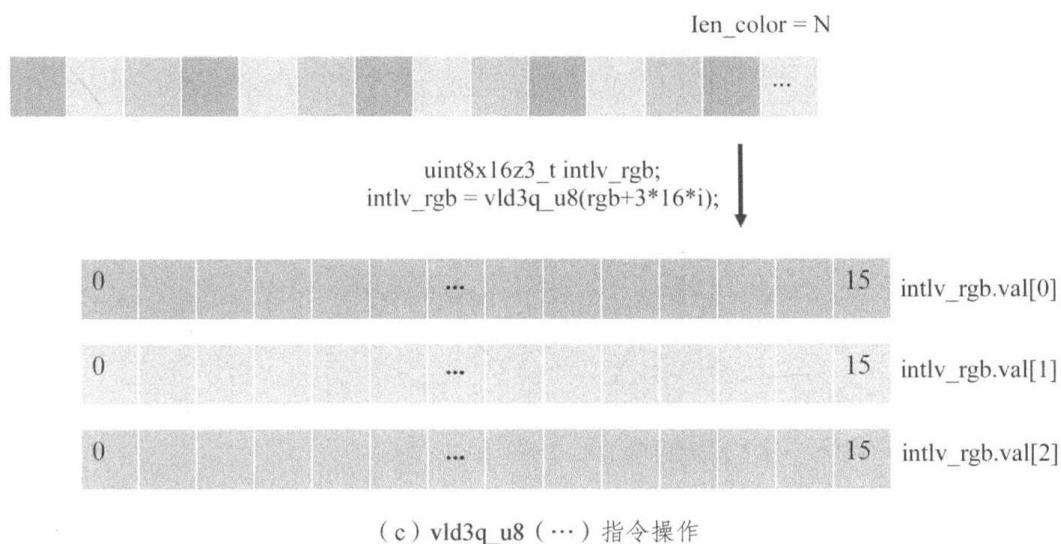
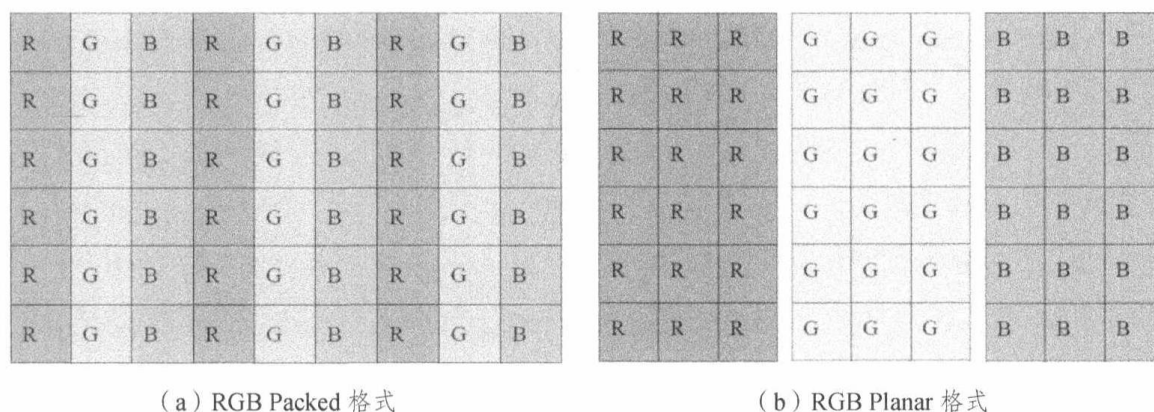


图 8-19 RGB 内存数据排列顺序解交织

完整测试代码如下：

```

// #if __ARM_NEON

// #if __ARM_NEON
#include <arm_neon.h>
// #endif // __ARM_NEON

#include <stdlib.h>
#include <stdio.h>
#include <memory.h>
// #include <fstream>

#include <sys/time.h>

// code from https://developer.arm.com/documentation/102467/0100/Example---RGB-
// deinterleaving
void rgb_deinterleave_c ( uint8_t *r, uint8_t *g, uint8_t *b, uint8_t *rgb, int len_color )
{
    /* Take the elements of "rgb" and store the individual colors "r", "g", and "b" .*/
    for( int i=0; i < len_color; i++ )
    {
        r [ i ] = rgb [ 3*i ];
        g [ i ] = rgb [ 3*i+1 ];
        b [ i ] = rgb [ 3*i+2 ];
    }
}

void rgb_deinterleave_neon ( uint8_t *r, uint8_t *g, uint8_t *b, uint8_t *rgb, int len_
color )
{
    /* Take the elements of "rgb" and store the individual colors "r", "g", and "b" .*/
    int num8x16 = len_color / 16;
    // len_color 的值等于 W*H*3，为什么除以 16 呢，因为 Neon 寄存器的宽度为 128bit，
    // 也就是 16 个 Byte，
    // 这个向量寄存器一次可以加载 16 个字节。
    uint8x16x3_t intlv_rgb;
    for( int i=0; i < num8x16; i++ )

```



```

{
    intlv_rgb = vld3q_u8 ( rgb+3*16*i );
    // vld3q_u8 函数实现一次加载 3 个大小为 16 个字节的向量
    vst1q_u8 ( r+16*i, intlv_rgb.val [ 0 ] );
    // vst1q_u8 函数实现一次存储大小为 16 个字节的向量
    vst1q_u8 ( g+16*i, intlv_rgb.val [ 1 ] );
    vst1q_u8 ( b+16*i, intlv_rgb.val [ 2 ] );
}
}
//

int W = 960;
int H = 540;
struct timeval time_begin, time_end;
unsigned char *RGBRGBRGB_buf;
unsigned char *RRRGGBBB_buf;

int main ( int argc, char **argv )
{
    int pixel_size=W*H;
    int img_size=W*H*3;
    RGBRGBRGB_buf= ( unsigned char * ) malloc ( img_size );
    RRRGGBBB_buf= ( unsigned char * ) malloc ( img_size );

    for( int i = 0; i < pixel_size; i++ )
    {
        RGBRGBRGB_buf [ 3*i+0 ] =10;
        RGBRGBRGB_buf [ 3*i+1 ] =128;
        RGBRGBRGB_buf [ 3*i+2 ] =200;
    }
    //std::fstream file1 ( " ./RGBRGBRGB_buf.dat" , std::ios::binary|std::ios::out );
    //file1.write ( ( char * ) RGBRGBRGB_buf, img_size );
    //file1.close ( );

    int time = 0;
    // c time cost

```

```
gettimeofday ( &time_begin, NULL );

    rgb_deinterleave_c ( RRRGGG BBB_buf, RRRGGG BBB_buf + pixel_size,
RRRGGG BBB_buf + 2*pixel_size, RGBRGBRGB_buf, pixel_size );
    gettimeofday ( &time_end, NULL );
    time = ( 1000000* ( time_end.tv_sec-time_begin.tv_sec ) + ( time_end.tv_usec-time_
begin.tv_usec ) ) /1000;
    printf ( " c cost: %d ms\n" , time );

    //std::fstream file2 ( " ./RRRGGG BBB_buf_c.dat" , std::ios::binary|std::ios::out );
    //file2.write ( ( char * ) RRRGGG BBB_buf, img_size );
    //file2.close ( );

    // neon time cost
    gettimeofday ( &time_begin, NULL );
    rgb_deinterleave_neon ( RRRGGG BBB_buf, RRRGGG BBB_buf + pixel_size,
RRRGGG BBB_buf + 2*pixel_size, RGBRGBRGB_buf, pixel_size );
    gettimeofday ( &time_end, NULL );
    time = ( 1000000* ( time_end.tv_sec-time_begin.tv_sec ) + ( time_end.tv_usec-time_
begin.tv_usec ) ) /1000;
    printf ( " neon cost: %d ms\n" , time );

    //std::fstream file3 ( " ./RRRGGG BBB_buf_neon.dat" , std::ios::binary|std::ios::out );
    //file3.write ( ( char * ) RRRGGG BBB_buf, img_size );
    //file3.close ( );

    return 0;
}
```

在 RV1126 平台的运行性能评估如下：

(1) RV1126 平台 CPU 测试

编译：

```
$ arm-linux-gnueabihf-g++ neon_deinterleave.c -g -o test_neon
```

运行结果如表 8-31 所示：

表 8-31 RV1126 平台 CPU 运行性能对比

RV1126 CPU	pure c cost	Neon cost
默认编译配置	24 ms	5 ms
编译中配置 -O3 优化参数	3 ms	4 ms

(2) V853 平台 CPU (ARM Cortex-A7@1GHz, 32 KB I-cache, 32 KB D-cache, 128 KB L2 cache) 测试

编译和测试:

```
$ /V853/tina-v853-open/prebuilt/rootfsbuilt/arm/toolchain-sunxi-musl-gcc-830/toolchain/
bin/arm-openwrt-linux-gcc -o test_neon_deinterleave test_neon_deinterleave.c
```

```
$ adb push test_neon_deinterleave home/test
```

运行结果如表 8-32 所示:

表 8-32 V853 平台 CPU 运行性能对比

V853 平台 CPU	pure c cost	Neon cost
默认编译配置	40 ms	4 ms
编译中配置 -O3 优化参数	8 ms	6 ms

8.5.3 矩阵乘法性能优化

关于矩阵乘法的 C 语言实现, 其代码如下^①:

```
void matrix_multiply_c ( float32_t *A, float32_t *B, float32_t *C, uint32_t n, uint32_t
m, uint32_t k )
{
    for( int i_idx=0; i_idx<n; i_idx++ )
    {
        for( int j_idx=0; j_idx<m; j_idx++ )
        {
            C [ n*j_idx + i_idx ] = 0;
            for( int k_idx=0; k_idx<k; k_idx++ )
            {
                C [ n*j_idx + i_idx ] +=
                    A [ n*k_idx + i_idx ] *B [ k*j_idx + k_idx ];
            }
        }
    }
}
```

^① 出自: <https://developer.arm.com/documentation/102467/0201/Example---matrix-multiplication>

接下来我们讨论如何使用 Neon 来优化矩阵乘法。float32x4_t 类型的 Neon 寄存器中的向量通道为 4，每个通道数据类型为 float32。我们将大尺寸矩阵分解为多个 4×4 小矩阵，再优化 4×4 小矩阵乘，矩阵乘的流程如图 8-20 所示。细心的读者朋友会发现和线性代数里的流程有点不同，但对编程实现来说是等效的。

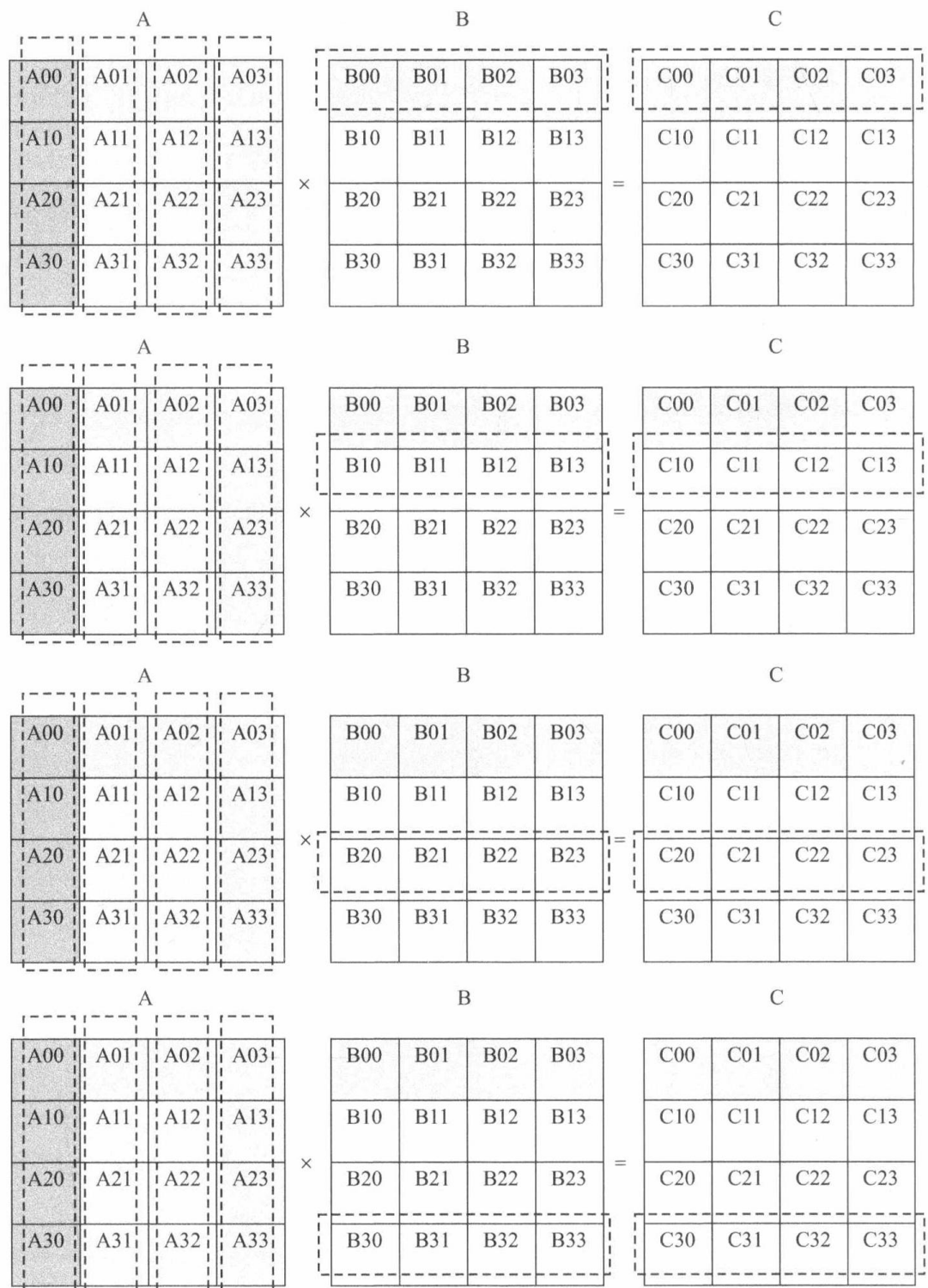


图 8-20 矩阵 A 乘矩阵 B 输出矩阵示意图

矩阵乘的计算方式如下：

$$C00 = B00 \times A00 + B01 \times A10 + B02 \times A20 + B03 \times A30$$

$$C01 = B00 \times A01 + B01 \times A11 + B02 \times A21 + B03 \times A31$$

$$C02 = B00 \times A02 + B01 \times A12 + B02 \times A22 + B03 \times A32$$

$$C03 = B00 \times A03 + B01 \times A13 + B02 \times A23 + B03 \times A33$$

公式可以变换为

$$C00 = (A00, A10, A20, A30) \odot (B00, B01, B02, B03)$$

$$C01 = (A01, A11, A21, A31) \odot (B00, B01, B02, B03)$$

$$C02 = (A02, A12, A22, A32) \odot (B00, B01, B02, B03)$$

$$C03 = (A03, A13, A23, A33) \odot (B00, B01, B02, B03)$$

其中 $\odot$ 代表点积 (dot product)。

$$A0 = (A00, A10, A20, A30)$$

$$A1 = (A01, A11, A21, A31)$$

$$A2 = (A02, A12, A22, A32)$$

$$A3 = (A03, A13, A23, A33)$$

$$B0 = (B00, B01, B02, B03)$$

通过 Neon 指令实现：

$$C0 = \text{vfmaq_laneq_f32} (C0, A0, B0, 0);$$

$$C0 = \text{vfmaq_laneq_f32} (C0, A1, B0, 1);$$

$$C0 = \text{vfmaq_laneq_f32} (C0, A2, B0, 2);$$

$$C0 = \text{vfmaq_laneq_f32} (C0, A3, B0, 3);$$

vfmaq_laneq_f32 ^① 指令实现单指令周期里一次乘加运算，只有 Armv8-A、Armv9-A 架构等支持。在 Armv7-A 架构 CPU 中可以将 vfmaq_laneq_f32 指令通过 vgetq_lane_f32 、 vmulq_n_f32 、 vaddq_f32 指令组合实现，当然这样性能会有影响。将

$$C0 = \text{vfmaq_laneq_f32} (C0, A0, B0, 0);$$

$$C0 = \text{vfmaq_laneq_f32} (C0, A1, B0, 1);$$

$$C0 = \text{vfmaq_laneq_f32} (C0, A2, B0, 2);$$

$$C0 = \text{vfmaq_laneq_f32} (C0, A3, B0, 3);$$

修改为：

$$C0 = \text{vaddq_f32} (\text{vmulq_n_f32} (A0, \text{vgetq_lane_f32} (B0, 0)), C0);$$

$$C0 = \text{vaddq_f32} (\text{vmulq_n_f32} (A1, \text{vgetq_lane_f32} (B0, 1)), C0);$$

$$C0 = \text{vaddq_f32} (\text{vmulq_n_f32} (A2, \text{vgetq_lane_f32} (B0, 2)), C0);$$

① Vector Fused Multiply Accumulate, 融合乘加运算

C0 = vaddq_f32 (vmulq_n_f32 (A3, vgetq_lane_f32 (B0, 3)), C0);
其他位置的代码修改同理。

其中 vmulq_n_f32 指令实现向量浮点数乘以常数 (Scalar), 定义如下:

Result_t vmulq_n_type (Vector_t N, Scalar_t M);

vaddq_f32 指令实现向量浮点数加法, 定义如下:

Result_t vaddq_type (Vector1_t N, Vector2_t M);

代码编译:

\$ arm-linux-gnueabi-g++ matrix-multiplication-neon.cpp -O3 -g -o test_neon

运行结果如表 8-33 所示:

表 8-33 运行性能对比

RV1126 CPU 矩阵大小均为 400 × 400	pure c cost	Neon cost
编译中配置 -O3 优化参数	1141 ms	297 ms

8.6 NEON2SSE 介绍

NEON2SSE 是 Intel 提供的用于将 NEON 内联函数转化为 Intel SIMD-SSE 内联函数的接口。通过 NEON2SSE 可实现基于 X86 平台开发 SIMD 程序, 还可以实现在 X86 平台上调试 ARM 平台的 NEON 程序。NEON2SSE 使用起来简单, 只需在我们的代码中添加以下头文件就可以。

```
#ifdef ARM_PLATFORM
#include <arm_neon.h>
#else
#include "NEON_2_SSE.h"
#endif
// https://github.com/intel/ARM_NEON_2_x86_SSE
```


第 9 章

基于 TFLite 的端侧模型部署 和性能优化

本章主要讨论如何使用 TFLite (TensorFlow Lite) 框架实现端侧模型的部署, 其内容包括 TFLite 交叉编译、TFLite FlatBuffer 模型文件转换和验证、端侧模型部署和性能评估以及性能优化。本章中用于测试的模型包括 YOLOv8 和 Transformer。TFLite 方案相比较其他方案, 其中一个显著优势是完全开源。

9.1 TFLite 委托

TFLite 委托 (Delegate) 是指在模型部署中利用设备端的加速器 (例如 GPU、DSP、NPU 等) 来运行 TFLite 模型的部分或全部网络。通过委托 API 去调用设备中加速器的 API, 让开发更加简单。TFLite 包括转换 (Converter) 和解析 (Interpreter) 两部分, 前者用于 TFLite FlatBuffer 模型文件转换, 包括优化和量化, 后者用于模型加载、Buffer 分配、推理和委托等, 如图 9-1 所示。

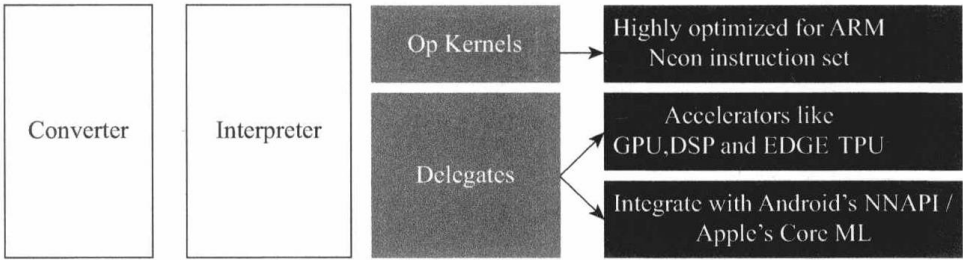


图 9-1 TFLite 委托流程^①

9.2 TFLite 交叉编译和部署——基于 ARM 平台

我们的需求是把一个训练好的模型部署在 ARM 平台, 并且性能满足需求。首先使用命令 `cat /proc/cpuinfo` 查看部署平台 CPU 的详细信息, 包括 CPU 型号、核数、线程数、CPU 主频以及缓存信息等。以 RV1126 平台为例, 进入平台终端后, 运行以上命令得到的信息如图 9-2 所示, 可以知道该平台具有 4 个 ARM Cortex-A7 核, 且每个核均支持 Neon。

^① 图来源: <https://tensorflow.google.cn/lite/performance/delegates?hl=zh-cn>; https://tensorflow.google.cn/lite/performance/implementing_delegate?hl=zh-cn

```
[root@ATK-DLRV1126:/demo]# cat /proc/cpuinfo
processor       : 0
model name     : ARMv7 Processor rev 5 (v7l)
BogoMIPS      : 121.00
Features      : half thumb fastmult vfp edsp neon vfpv3 tls vfpv4 idiva idivt vfpd32 lpae
processor       : 1
model name     : ARMv7 Processor rev 5 (v7l)
BogoMIPS      : 121.00
Features      : half thumb fastmult vfp edsp neon vfpv3 tls vfpv4 idiva idivt vfpd32 lpae
processor       : 2
model name     : ARMv7 Processor rev 5 (v7l)
BogoMIPS      : 121.00
Features      : half thumb fastmult vfp edsp neon vfpv3 tls vfpv4 idiva idivt vfpd32 lpae
processor       : 3
model name     : ARMv7 Processor rev 5 (v7l)
BogoMIPS      : 121.00
Features      : half thumb fastmult vfp edsp neon vfpv3 tls vfpv4 idiva idivt vfpd32 lpae
```

图 9-2 CPU 的详细信息 (图有改动)

9.2.1 用 CMake 工具构建 TFLite

第 1 步, 克隆 TensorFlow 仓库。

```
git clone https://github.com/tensorflow/tensorflow.git
```

其版本信息如下:

```
tensorflow$ git log
```

```
commit 63f5a65c7cd7b6241bede8d2e0082058566ea364 (HEAD, tag: v2.15.1, origin/
r2.15)
```

第 2 步, 创建 CMake 构建目录。

```
mkdir build_arm
```

```
cd build_arm
```

第 3 步, 使用配置运行 CMake 工具。

```
tensorflow/build_arm$
```

```
ARMCC_FLAGS="-march=armv7-a -mfpv=neon-vfpv4 -funsafe-math-optimizations
-mfp16-format=ieee"
```

```
ARMCC_PREFIX=/opt/atk-dlr1126-toolchain/bin/arm-linux-gnueabihf-
```

```
cmake -DCMAKE_C_COMPILER=${ARMCC_PREFIX}gcc \
```

```
-DCMAKE_CXX_COMPILER=${ARMCC_PREFIX}g++ \
```

```
-DCMAKE_C_FLAGS=" ${ARMCC_FLAGS}" \
```

```
-DCMAKE_CXX_FLAGS=" ${ARMCC_FLAGS}" \
```

```
-DCMAKE_VERBOSE_MAKEFILE:BOOL=ON \
```

```
-DCMAKE_SYSTEM_NAME=Linux \
```

```
-DCMAKE_SYSTEM_PROCESSOR=armv7 \
```

```
../tensorflow/lite/
```

先说明一下以上的多个编译选项，具体内容如下：

`-march=armv7-a` 表示编译出的目标文件是针对 Armv7 架构。

`-mfpu=neon-vfpv4` 用于指定目标处理器的浮点单元（FPU）类型。其中 `neon` 表示使用 Neon 指令集进行加速运算，`vfpv4` 表示使用 VFPv4 指令集进行浮点数运算。这个选项通常用于支持 ARM 架构的处理器，以优化浮点数运算的性能。

`-funsafe-math-optimizations`，用于启用 GCC 编译器的不安全数学优化。当启用此选项时，编译器会尝试对数学运算进行优化，以提高程序的性能，但这可能导致某些情况下的数值精度损失。如果需要确保数值精度，建议禁用此选项。

`-mfp16-format=ieee` 用于指定浮点数的表示格式。`mfp16` 表示使用半精度浮点数（Half-precision Floating Point），`ieee` 表示使用 IEEE 754 标准来表示浮点数。这个选项通常用于嵌入式系统或需要节省存储空间场景。

`-DCMAKE_VERBOSE_MAKEFILE:BOOL=ON` 表示 CMake 构建过程中输出详细的编译命令、链接命令等构建过程信息。

`-DCMAKE_CXX_COMPILER` 指定 C++ 编译器的路径，`-DCMAKE_C_FLAGS` 指定 C 编译器的路径。

`-DCMAKE_SYSTEM_PROCESSOR=armv7` 指定目标系统的处理器架构是 ARM 架构 32 位处理器。

如果在配置运行构建中出现如下错误：

```
/tensorflow/build_arm/xnnpack/src/qs8-qc8w-igemm/gen/qs8-qc8w-igemm-2x2c4-
minmax-fp32-armsimd32.c:80:15: error: unknown type name ' int16x2_t'
```

```
    const int16x2_t valc02 = __sxtb16 ( val );
```

```
.....
```

```
/tensorflow/build_arm/xnnpack/src/qs8-qc8w-igemm/gen/qs8-qc8w-igemm-2x2c4-
minmax-fp32-armsimd32.c:83:24: error: expected ' = ' , ' , ' , ' ; ' , ' asm ' or ' __
attribute__ ' before ' vb0'
```

```
    const int8x4_t vb0 = * (( const int8x4_t* ) w ); w = ( const int8_t* ) w + 4;
```

```
/tensorflow/build_arm/xnnpack/src/qs8-qc8w-igemm/gen/qs8-qc8w-igemm-2x2c4-
minmax-fp32-armsimd32.c:83:24: error: ' vb0 ' undeclared ( first use in this function ); did
you mean ' va0 ' ?
```

```
    const int8x4_t vb0 = * (( const int8x4_t* ) w ); w = ( const int8_t* ) w + 4;
```

```
    va0
```

```
.....
```

```
/tensorflow/build_arm/xnnpack/src/qs8-qc8w-igemm/gen/qs8-qc8w-igemm-2x2c4-
minmax-fp32-armsimd32.c:83:30: error: invalid type argument of unary ' * ' ( have ' int ' )
```